claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-a677e6345f2fa8b53.jsonl`
- SHA-256: `738cd94a673cc67ed576e645d02727a1698d769d3d145fec02f720371a3b0681`
- Source modified: `2026-06-16T06:30:29+00:00`
- Imported at: `2026-07-05T16:48:29+00:00`
- project: `wf_ef85c152-821`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`

Transcript

1. user (2026-06-16T06:26:59.416Z)

LENS = ROBUSTNESS to the concurrent quality track + future change. The fixtures pin a windowing preset (vt/mg/mwd/max_win) as a LITERAL, not the named SERVED constant. If a future PR changes the build_windows / trajectory_to_action_windows ALGORITHM (the windowing session shipped #187 bounded-windowing, #188/#190 net-crossings gate), does the characterization fail LOUDLY (good) or could it pass silently / error? Does the gate/windowing change interact with the CONTROL path at all? If distill_local.build_candidates or rally_gate signatures change, what breaks and is it caught? Is the pinned-literal-vs-SERVED divergence documented + safe? What happens when real fixtures (P3) + corpus growth arrive (runtime, the deferred M4 quality-floor)?

AUDIT the SHIPPED 'Golden Regression Fixtures' feature (4 merged PRs #186/#189/#191/#192) for any gap the per-PR verification missed.
Read files in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (master HEAD 8d3e185): backend/eval/golden_fixtures.py (the core); tests/test_golden_fixtures.py; tests/test_golden_real_fixtures.py; eval_baselines/fixtures/ (3 synthetic fixtures + manifest.json + README.md); .github/workflows/ci.yml (the golden-crossos job); docs/GOLDEN_REGRESSION_FIXTURES.md (§4c/§5/§6 claims). For analysis also read backend/eval/{experiment,distill_local,metrics,gt_loader,regression}.py and backend/pipeline/detectors/{tracknet_runner,rally_gate}.py.
THE FEATURE: a video-free regression harness. replay_fixture = parse_trajectory_csv -> distill_local.build_candidates(PINNED preset) -> VideoCandidates(5 shuttle cols only) -> experiment.predict(CONTROL, pure-stdlib) -> metrics.score. Synthetic Tier-0 fixtures + characterization (parse-compare: exact on int/structural, approx 1e-6 on float).
freeze_real_fixture = de-attribution (random fx_<hex> slug, metadata strip, metric-invariant time-origin reset, STRICT gt_loader, positive no-person/audio/MD5/source-path scan) + the M2 refusal gate (raise unless owned AND minors_free AND non-blank license). The golden-crossos CI job runs `--check` + tests/test_golden_fixtures.py on ubuntu/windows/macos.
ALREADY VERIFIED ? do NOT re-report as new findings: full suite 759 pass/7 skip; coverage 80.49%; ruff+mypy clean; cross-OS green on all 3 OSes for the COMMITTED fixtures; CONTROL avoids numpy *execution*.
Find what that MISSED: real gaps / bugs / edge-cases / privacy holes / determinism holes / over-claims. Cite file:line. Do NOT invent issues ? if a lens is clean, say so honestly and set verdict accordingly.

2. assistant (2026-06-16T06:27:02.570Z)

I'll audit the shipped Golden Regression Fixtures feature. Let me start by reading the core files in the worktree.

3. user (2026-06-16T06:27:04.286Z)

8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate (#192)
On branch feat/golden-fixtures-doc

Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

4. user (2026-06-16T06:27:04.908Z)

```
"""Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.

`docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
replay code path** used by both the freeze tool and the characterization gate, and it
deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we replay
only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate intervals,
merges overlaps, and uses **no numpy**.

The replay reuses the already-shipped eval stack VERBATIM::

    parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
        ? VideoCandidates(name, intervals, features={5 shuttle cols}, gts=load_gt_intervals)
        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)

Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:

* Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry zero
  provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
* Every fixture is tagged ``kind="synthetic"``, ``minors_free=true``, ``license="synthetic"``.
* ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by design, no
  person / pose / gait / face / audio field anywhere in the schema (positive-assertion test).
* The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each fixture's
  provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the named
  ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn fixtures.

§6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
suite proves only that the deterministic post-trajectory transforms are unchanged for the
frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and NOT
that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
field quality (never cite them in QUALITY_ITERATIONS.md / the paper).

Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-only
for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
"""
from __future__ import annotations

import argparse
import json
import math
import os
import re
import sys
from dataclasses import dataclass
from pathlib import Path
```

```python
from typing import Any, Dict, List, Optional, Sequence, Tuple

# --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only) -------------
from backend.eval import distill_local
from backend.eval.experiment import CONTROL, VideoCandidates, predict
from backend.eval.gt_loader import load_gt_intervals
from backend.eval.metrics import score
from backend.eval.regression import gt_hash
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

Interval = Tuple[float, float]

#: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
#: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
CURRENT_SCHEMA = 1

#: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness works
#: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ? root.
FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"

#: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh, merge_gap,
#: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a LITERAL
#: here on purpose ? the spec forbids depending on the named ``windowing.SERVED`` constant so a
#: future retune of SERVED can never silently re-baseline a committed fixture. Each fixture also
#: re-states these in its provenance; the replay reads them from there.
PINNED_VT = 0.02
PINNED_MERGE_GAP = 2.0
PINNED_MIN_WINDOW = 2.0
PINNED_MAX_WINDOW = 0.0  # 0 = bounded-windowing OFF (W1), matching the SERVED default

#: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-stated
#: here (not imported as a mutable alias) so a fixture's expected.json is pinned to exactly these.
SHUTTLE_FEATURES: Tuple[str, ...] = (
    "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
)
assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
    "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
)

#: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT appear
#: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")

_FLOAT_ROUND_DP = 6  # all written floats rounded to this many decimals (determinism)
_FLOAT_TOL = 1e-6    # abs tolerance when parse-comparing recomputed vs committed floats

#: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan refuses to
#: emit a fixture whose written text contains one (it would be an attribution back-channel, §4c/§5).
_MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")


# ==============================================================================
# Synthetic trajectory generation (deterministic ? NO random)
```

```
# ==========================================================================================


@dataclass(frozen=True)
class RallySpec:
    """One in-flight rally segment: the shuttle oscillates across the net midline.

    ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
    period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
    net midline along the play axis. Deterministic sine motion ? fixed velocity/crossings.
    """

    start_frame: int
    n_frames: int
    period: float = 30.0
    amplitude: float = 400.0


@dataclass(frozen=True)
class GapSpec:
    """Dead-time between rallies ? what separates one window from the next.

    ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
    ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
    end an active run so the next rally forms a distinct window.
    """

    start_frame: int
    n_frames: int
    kind: str = "still"  # "still" | "occluded"


@dataclass(frozen=True)
class FixtureSpec:
    """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

    ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
    "labels.csv". For a synthetic fixture they line up with the rallies by construction,
    which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
    field-quality one (§6).
    """

    slug: str
    description: str
    fps: float
    frame_width: float
    frame_height: float
    segments: Tuple[Any, ...]  # ordered (RallySpec | GapSpec)
    gts: Tuple[Interval, ...]
    net_midline: float = 640.0


def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
                  ) -> List[Tuple[int, int, float, float]]:
    """Deterministic in-flight shuttle: sine oscillation across the net midline (play axis = x)."""
    rows: List[Tuple[int, int, float, float]] = []
    y_mid = frame_height / 2.0
    for i in range(spec.n_frames):
        x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
        # A small orthogonal wobble keeps y-spread < x-spread so the play axis is unambiguously x.
        y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
        rows.append((spec.start_frame + i, 1, x, y))
```

```python
    return rows


def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
                ) -> List[Tuple[int, int, float, float]]:
    """Dead-time rows: resting-visible (still) or absent (occluded)."""
    rows: List[Tuple[int, int, float, float]] = []
    y_mid = frame_height / 2.0
    for i in range(spec.n_frames):
        if spec.kind == "occluded":
            rows.append((spec.start_frame + i, 0, 0.0, 0.0))
        elif spec.kind == "still":
            rows.append((spec.start_frame + i, 1, net_midline, y_mid))
        else:  # pragma: no cover - guarded by builder
            raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'|'occluded')")
    return rows


def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
    """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic scenario."""
    rows: List[Tuple[int, int, float, float]] = []
    for seg in spec.segments:
        if isinstance(seg, RallySpec):
            rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
        elif isinstance(seg, GapSpec):
            rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
        else:  # pragma: no cover - guarded by dataclass shape
            raise TypeError(f"unknown segment type {type(seg).__name__}")
    rows.sort(key=lambda r: r[0])
    return rows


def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) -> None:
    """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp floats."""
    lines = ["Frame,Visibility,X,Y"]
    for frame, vis, x, y in rows:
        lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
    path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
    """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format), LF."""
    lines = ["start,end"]
    for s, e in gts:
        lines.append(f"{_fmt(s)},{_fmt(e)}")
    path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")


# ==================================================================================
# Built-in synthetic scenarios (the 3 committed slugs)
# ==================================================================================


def _fr(seconds: float, fps: float) -> int:
    return int(round(seconds * fps))


def builtin_specs() -> List[FixtureSpec]:
    """The committed synthetic scenarios. Deterministic; chosen so each exercises a distinct
    windowing behaviour (one window / dead-time split / occlusion split)."""
```

```
    fps = 30.0
    fw, fh = 1280.0, 720.0
    net = 640.0

    # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window, crossings>0.
    single = FixtureSpec(
        slug="clean_single_rally",
        description="One continuous ~4s rally; shuttle oscillates across the net ? 1 window.",
        fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
        segments=(RallySpec(start_frame=0, n_frames=120),),
        # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
        gts=((0.0, 4.0),),
    )

    # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split by the
    #    low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
    r0 = RallySpec(start_frame=0, n_frames=120)
    gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
    r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
    multi = FixtureSpec(
        slug="multi_rally_with_deadtime",
        description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-time ? 2 windows.",
        fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
        segments=(r0, gap_still, r1),
        gts=((0.0, 4.0), (7.0, 11.0)),
    )

    # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by the
    #    visibility break ? proves occlusion (not just low velocity) separates windows.
    o0 = RallySpec(start_frame=0, n_frames=120)
    gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
    o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
    occ = FixtureSpec(
        slug="occlusion_break",
        description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0) ? 2 windows.",
        fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
        segments=(o0, gap_occ, o1),
        gts=((0.0, 4.0), (7.0, 11.0)),
    )

    return [single, multi, occ]


def _spec_by_slug() -> Dict[str, FixtureSpec]:
    return {s.slug: s for s in builtin_specs()}


# =====================================================================================
# The shared replay (CONTROL / pure-stdlib path)
# =====================================================================================


def _fmt(x: float) -> str:
    """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to ``0.0``."""
    r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
    if r == 0.0:
        r = 0.0
    return f"{r:.6f}"
```

```python


def _round(x: float) -> float:
    r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
    return 0.0 if r == 0.0 else r


def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
    """The windowing proposal tuple, read from the fixture provenance when present (the pinned
    params travel with the fixture), else the module-level pinned defaults. NEVER windowing.SERVED."""
    if prov and "windowing" in prov:
        w = prov["windowing"]
        return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
    return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)


def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
    """Replay one fixture through the shipped CONTROL path; return its recomputed snapshot.

    Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its committed
    ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

        parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5 shuttle
        cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).

    Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
    control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate then
    merges overlaps; NO numpy is touched.
    """
    fdir = _resolve_dir(slug_or_dir)
    slug = fdir.name
    traj_csv = fdir / "trajectory.csv"
    labels_csv = fdir / "labels.csv"
    prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file() else None

    fps, fw = _fps_fw(prov, slug)
    iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
    proposal = _pinned_proposal(prov)

    intervals, feature_rows = distill_local.build_candidates(
        str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)

    # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never call
    # fusion_compare.load_videos / ablation.load_videos ? those read c['person'] unconditionally
    # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
    features: Dict[str, List[float]] = {
        name: [row[i] for row in feature_rows] for i, name in enumerate(SHUTTLE_FEATURES)
    }
    gts = load_gt_intervals(str(labels_csv), strict=True)
    vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)

    preds = predict(CONTROL, vc)  # CONTROL: keep all ? merge_overlaps; pure-stdlib, no numpy
    sc = score(preds, gts, iou)

```

```
    candidates: List[Dict[str, Any]] = []
    for j, (s, e) in enumerate(intervals):
        shuttle = {name: _round(feature_rows[j][i]) for i, name in enumerate(SHUTTLE_FEATURES)}
        # net_crossings is conceptually an integer count ? keep it exact (int), per constraint 2.
        shuttle["net_crossings"] = int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
        candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})

    return {
        "slug": slug,
        "schema_version": CURRENT_SCHEMA,
        "iou": _round(iou),
        "candidates": candidates,
        "control_segments": [[_round(s), _round(e)] for s, e in preds],
        "score": {
            "iou_threshold": _round(sc.iou_threshold),
            "num_pred": int(sc.num_pred),
            "num_gt": int(sc.num_gt),
            "true_positives": int(sc.true_positives),
            "false_positives": int(sc.false_positives),
            "false_negatives": int(sc.false_negatives),
            "precision": _round(sc.precision),
            "recall": _round(sc.recall),
            "f1": _round(sc.f1),
            "mean_iou": _round(sc.mean_iou),
            "mean_start_error": _round(sc.mean_start_error),
            "mean_end_error": _round(sc.mean_end_error),
        },
        "gt_hash": gt_hash(gts),
    }


# ====================================================================================
# Freeze / refresh (expected.json + provenance.json are correct-by-construction)
# ====================================================================================


def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
                ) -> Dict[str, Any]:
    """The committed provenance block ? pins the windowing + privacy/licensing tags (§4c, §5)."""
    desc = ""
    if isinstance(spec_or_dir, FixtureSpec):
        desc = spec_or_dir.description
    return {
        "slug": slug,
        "schema_version": CURRENT_SCHEMA,
        "kind": "synthetic",
        "license": "synthetic",
        "minors_free": True,
        "description": desc,
        "fps": _round(fps),
        "frame_width": _round(fw),
        "iou": _round(iou),
        "windowing": {
            "vt": PINNED_VT,
            "mg": PINNED_MERGE_GAP,
            "mwd": PINNED_MIN_WINDOW,
            "max_win": PINNED_MAX_WINDOW,
        },
        "paths": {
            "trajectory": "trajectory.csv",
            "labels": "labels.csv",
```

```
            "expected": "expected.json",
        },
        "note": (
            "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
            "Plumbing-correctness fixture ? NOT a measure of rally-detection quality (§6)."
        ),
    }


def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
                   iou: float = 0.5) -> Dict[str, Any]:
    """Compute ``expected.json`` + ``provenance.json`` for one fixture from its COMMITTED
    ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-construction).

    Idempotent: rewrites the same bytes given the same committed inputs. Requires the trajectory
    + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
    """
    fdir = fixtures_dir / slug
    traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
    if not traj_csv.is_file() or not labels_csv.is_file():
        raise FileNotFoundError(
            f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to generate them")

    # Provenance first (it pins the windowing the replay reads), then replay ? expected.
    spec = _spec_by_slug().get(slug)
    fps, fw = (spec.fps, spec.frame_width) if spec is not None else _fps_fw_from_prov(fdir, slug)
    prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
    _write_json(fdir / "provenance.json", prov)

    expected = replay_fixture(fdir)
    _write_json(fdir / "expected.json", expected)
    return expected


def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
    """Generate trajectory.csv + labels.csv for every built-in scenario, freeze expected.json +
    provenance.json, and (re)write the manifest. Returns the slugs frozen."""
    specs = builtin_specs()
    fixtures_dir.mkdir(parents=True, exist_ok=True)
    slugs: List[str] = []
    for spec in specs:
        fdir = fixtures_dir / spec.slug
        fdir.mkdir(parents=True, exist_ok=True)
        rows = make_synthetic_trajectory(spec)
        write_trajectory_csv(rows, fdir / "trajectory.csv")
        write_labels_csv(spec.gts, fdir / "labels.csv")
        freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
        slugs.append(spec.slug)
    _write_manifest(specs, fixtures_dir)
    return slugs


def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
                     fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
    """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv + labels.csv ?
    NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real regression; the
```

```
    mitigation is reviewing the visible diff + recording a reason (the CLI requires one)."""
    if all_slugs:
        targets = [d.name for d in sorted(fixtures_dir.iterdir())
                   if d.is_dir() and (d / "trajectory.csv").is_file()]
    elif slug:
        targets = [slug]
    else:
        raise ValueError("refresh_snapshot needs --slug S or --all")
    for s in targets:
        freeze_fixture(s, fixtures_dir=fixtures_dir)
    return targets


# =========================================================================================
# P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
# fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
# §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
# =========================================================================================


class RefusalError(ValueError):
    """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.

    A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch it) that
    names *why* the unsafe public-fixture path was refused. The refusal is the code-enforced
    provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded / minors-excluded
    / license flags are human-asserted, but the gate makes the unsafe path impossible to reach
    *silently* ? a fixture cannot be written unless all three are affirmatively set.
    """


def _new_surrogate_slug() -> str:
    """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).

    Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a random,
    NON-reversible id with no algebraic link to the source, so it can't be brute-forced over a
    tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives off-git.
    """
    return f"fx_{os.urandom(8).hex()}"


#: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-path tokens
#: that collide with these are NOT a leak ? exclude them so a short source stem like ``traj`` can't
#: false-positive against the literal ``trajectory.csv``/``labels.csv``/``expected.json``.
_SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
    "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
    "provenance", "provenance.json",
)


def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
    """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything attributable.

```

```
        Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity substring,
        (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source path
        (full path / basename / stem) ? excluding the fixed schema filename literals the writers
        always emit. The trajectory is shuttle-only by nature; this guards against a *regression*
        re-introducing a person/audio/pose stream or an id/path back-channel ? never the silent default.
        """
        low = text.lower()
        for key in FORBIDDEN_KEYS:
            if key in low:
                raise RefusalError(
                    f"{where}: forbidden key substring {key!r} found in written output ? "
                    f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player MUST NOT "
                    f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
        m = _MD5_RE.search(text)
        if m:
            raise RefusalError(
                f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is the "
                f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
        for sp in source_paths:
            if not sp:
                continue
            p = Path(sp)
            for token in (sp, p.name, p.stem):
                t = (token or "").strip().lower()
                # Skip a token that is itself a substring of a schema-literal the writers always emit
                # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not a leak.
                if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
                    continue
                if t in low:
                    raise RefusalError(
                        f"{where}: source-path token {token!r} found in written output ? the source "
                        f"filename/path must never be persisted (§4c metadata strip).")


    def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
                               ) -> List[Tuple[int, int, float, float]]:
        """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ? ``(Frame,Visibility,X,Y)`` rows.

        Visibility/X/Y are passed through untouched (the shuttle geometry is metric-invariant under a
        pure time shift); only the absolute frame index moves toward 0, severing the match timeline.
        """
        rows: List[Tuple[int, int, float, float]] = []
        for p in points:
            rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x), float(p.y)))
        rows.sort(key=lambda r: r[0])
        return rows


    def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
        """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-invariant)."""
```

```
        return [(s - t0_sec, e - t0_sec) for s, e in gts]


    def freeze_real_fixture(
        label: str,
        trajectory_csv_path: Any,
        labels_csv_path: Any,
        *,
        license: str,
        minors_free: bool,
        owned: bool,
        fps: float,
        frame_width: float,
        fixtures_dir: Path = FIXTURES_DIR,
        slug: Optional[str] = None,
        time_origin_reset: bool = True,
        source_note: str = "",
    ) -> Dict[str, Any]:
        """Turn an OWNED, minors-free real trajectory + golden labels into a committable, de-attributed
        Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7 row P1.)

        The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture is emitted
        ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-blank string.
        The committed bytes carry NO filename / video_id / match / player / capture-date ? only an
        opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by default), the
        reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy assertion
        re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything attributable
        slipped in.

        NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-attributable
        only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v. SRB). De-
        attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off (P2) and the
        owner-asserted Fork-A / minors / biometric-exclusion flags.

        Returns the recomputed ``expected.json`` snapshot dict (identical to ``replay_fixture``).
        """
        # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?) -------
        reasons: List[str] = []
        if minors_free is not True:
            reasons.append("minors_free must be True (DPDP child = under 18; behavioural monitoring "
                           "of a minor is prohibited ? §3, guardrail 'exclude minors')")
        if owned is not True:
            reasons.append("owned must be True (only owner-recorded Fork-A source is safe to commit; "
                           "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
        if not (isinstance(license, str) and license.strip()):
            reasons.append("license must be a non-blank string (per-record provenance tag ? §4c "
                           "'provenance hard-fail gate')")
        if reasons:
            raise RefusalError(
```

```
                "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution does NOT "
                "cure provenance/licensing; this gate sits behind counsel sign-off, §6/P2):\n  - "
                + "\n  - ".join(reasons))

        traj_path = Path(str(trajectory_csv_path))
        labels_path = Path(str(labels_csv_path))

        # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source name ---
        out_slug = slug if slug is not None else _new_surrogate_slug()

        # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient) ----
        points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
        if not points:
            raise ValueError(f"{traj_path}: no parseable trajectory rows (Frame,Visibility,X,Y)")
        gts = load_gt_intervals(str(labels_path), strict=True)

        # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c) ------
        # Subtract the SAME offset from BOTH the trajectory frames and the label seconds: because
        # metrics.score is built from pure differences (interval_iou, |start-start|, |end-end|), the
        # score is bit-identical with vs without reset ? only the absolute timeline is severed. The
        # quality-floor / characterization snapshot is therefore unchanged (asserted in the tests).
        if time_origin_reset:
            t0_frame = min(int(p.frame) for p in points)
            t0_sec = t0_frame / float(fps)
        else:
            t0_frame, t0_sec = 0, 0.0
        rows = _reset_trajectory_rows(points, t0_frame)
        reset_gts = _reset_labels(gts, t0_sec)

        # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture) -----------
        fdir = fixtures_dir / out_slug
        fdir.mkdir(parents=True, exist_ok=True)
        write_trajectory_csv(rows, fdir / "trajectory.csv")
        write_labels_csv(reset_gts, fdir / "labels.csv")

        iou = 0.5
        prov = {
            "slug": out_slug,
            "schema_version": CURRENT_SCHEMA,
            "kind": "cleared_real",
            "label": str(label),
            "license": license,
            "minors_free": True,
            "owned": True,
            "fps": _round(fps),
            "frame_width": _round(frame_width),
            "iou": _round(iou),
            "time_origin_reset": bool(time_origin_reset),
            "windowing": {
                "vt": PINNED_VT,
                "mg": PINNED_MERGE_GAP,
                "mwd": PINNED_MIN_WINDOW,
                "max_win": PINNED_MAX_WINDOW,
            },
```

```
        "paths": {
            "trajectory": "trajectory.csv",
            "labels": "labels.csv",
            "expected": "expected.json",
        },
        "gt_hash": gt_hash(reset_gts),
        "source_note": str(source_note),
        "note": (
            "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded, biometric-"
            "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset. NON-"
            "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never 'anonymous'. "
            "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind counsel (P2)."
        ),
    }
    _write_json(fdir / "provenance.json", prov)

    expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction snapshot
    _write_json(fdir / "expected.json", expected)

    # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, §4c / ?) -----
    source_paths = [str(traj_path), str(labels_path)]
    _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
                    where=f"{out_slug}/expected.json", source_paths=source_paths)
    _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
                    where=f"{out_slug}/provenance.json", source_paths=source_paths)

    # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic) -----
    _append_manifest_entry(prov, fixtures_dir)
    return expected


def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
    """A manifest row for a single fixture, derived from its provenance block."""
    slug = prov["slug"]
    return {
        "slug": slug,
        "fps": _round(float(prov["fps"])),
        "frame_width": _round(float(prov["frame_width"])),
        "windowing": dict(prov.get("windowing", {})),
        "kind": prov.get("kind", "synthetic"),
        "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
        "paths": {
            "dir": slug,
            "trajectory": f"{slug}/trajectory.csv",
            "labels": f"{slug}/labels.csv",
            "expected": f"{slug}/expected.json",
            "provenance": f"{slug}/provenance.json",
        },
    }


def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
    """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g. synthetic)."""
    entries = load_manifest(fixtures_dir)
    entries = [e for e in entries if e.get("slug") != prov["slug"]]
    entries.append(_manifest_entry_for(prov))
    entries.sort(key=lambda e: str(e.get("slug")))
```

```
    _write_json(fixtures_dir / "manifest.json", entries)


# =====================================================================================
# Manifest + loaders
# =====================================================================================


def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
    entries = [{
        "slug": spec.slug,
        "fps": _round(spec.fps),
        "frame_width": _round(spec.frame_width),
        "windowing": {
            "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
            "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
        },
        "kind": "synthetic",
        "schema_version": CURRENT_SCHEMA,
        "paths": {
            "dir": spec.slug,
            "trajectory": f"{spec.slug}/trajectory.csv",
            "labels": f"{spec.slug}/labels.csv",
            "expected": f"{spec.slug}/expected.json",
            "provenance": f"{spec.slug}/provenance.json",
        },
    } for spec in specs]
    _write_json(fixtures_dir / "manifest.json", entries)


def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
    """Load the fixtures manifest (returns [] if absent)."""
    path = fixtures_dir / "manifest.json"
    if not path.is_file():
        return []
    data = _read_json(path)
    if not isinstance(data, list):  # pragma: no cover - corrupt manifest
        raise ValueError(f"{path}: manifest must be a JSON array")
    return data


def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
    """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses a fixture
    whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
    fdir = fixtures_dir / slug
    prov = _read_json(fdir / "provenance.json")
    expected = _read_json(fdir / "expected.json")
    sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
    if sv > CURRENT_SCHEMA:
        raise ValueError(
            f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade the harness")
    return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}


# =====================================================================================
# Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
# =====================================================================================


def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) -> List[str]:
    """Return a list of human-readable mismatch messages (empty == match).

```

```
    Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count, net_crossings,
    TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be EXACTLY
    equal; float fields compared with abs tolerance 1e-6. Never a byte compare ? CRLF/float-format
    safe (the repo runs with core.autocrlf=true)."""
    out: List[str] = []

    def exact(path: str, a: Any, b: Any) -> None:
        if a != b:
            out.append(f"{path}: {a!r} != {b!r} (exact)")

    def approx(path: str, a: Any, b: Any) -> None:
        try:
            if abs(float(a) - float(b)) > _FLOAT_TOL:
                out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
        except (TypeError, ValueError):
            out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")

    exact("slug", recomputed.get("slug"), committed.get("slug"))
    exact("schema_version", recomputed.get("schema_version"), committed.get("schema_version"))
    exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
    approx("iou", recomputed.get("iou"), committed.get("iou"))

    rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
    if len(rc) != len(cc):
        out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
    else:
        for i, (r, c) in enumerate(zip(rc, cc)):
            approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
            approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
            rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
            if set(rs) != set(cs):
                out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)} (exact)")
            else:
                for k in rs:
                    if k == "net_crossings":
                        exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
                    else:
                        approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])

    rseg, cseg = recomputed.get("control_segments", []), committed.get("control_segments", [])
    if len(rseg) != len(cseg):
        out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
    else:
        for i, (r, c) in enumerate(zip(rseg, cseg)):
            approx(f"control_segments[{i}][0]", r[0], c[0])
            approx(f"control_segments[{i}][1]", r[1], c[1])

    rsc, csc = recomputed.get("score", {}), committed.get("score", {})
    int_fields = ("num_pred", "num_gt", "true_positives", "false_positives", "false_negatives")
    float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
                    "mean_start_error", "mean_end_error")
    for k in int_fields:
        exact(f"score.{k}", rsc.get(k), csc.get(k))
    for k in float_fields:
        approx(f"score.{k}", rsc.get(k), csc.get(k))

    return out
```

```python

# ==========================================================================================
# Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
# ==========================================================================================


def _write_json(path: Path, obj: Any) -> None:
    text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
    path.write_text(text + "\n", encoding="utf-8", newline="\n")


def _read_json(path: Path) -> Any:
    with open(path, encoding="utf-8") as f:
        return json.load(f)


def _resolve_dir(slug_or_dir: Any) -> Path:
    if isinstance(slug_or_dir, Path):
        return slug_or_dir
    p = Path(str(slug_or_dir))
    if p.is_dir():
        return p
    return FIXTURES_DIR / str(slug_or_dir)


def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
    if prov and "fps" in prov and "frame_width" in prov:
        return float(prov["fps"]), float(prov["frame_width"])
    spec = _spec_by_slug().get(slug)
    if spec is not None:
        return spec.fps, spec.frame_width
    raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in spec")


def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
    prov_path = fdir / "provenance.json"
    prov = _read_json(prov_path) if prov_path.is_file() else None
    return _fps_fw(prov, slug)


# ==========================================================================================
# CLI
# ==========================================================================================


def _cmd_check(fixtures_dir: Path) -> int:
    manifest = load_manifest(fixtures_dir)
    if not manifest:
        print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first", file=sys.stderr)
        return 1
    failures = 0
    for entry in manifest:
        slug = entry["slug"]
        committed = _read_json(fixtures_dir / slug / "expected.json")
        recomputed = replay_fixture(fixtures_dir / slug)
        mismatches = compare_expected(recomputed, committed)
        if mismatches:
            failures += 1
            print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
                  f"(characterization = behaviour-locking, NOT correctness ? §6):")
            for m in mismatches:
                print(f"    - {m}")
        else:
```

```
                sc = recomputed["score"]
                print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
                      f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
        print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match "
              f"(synthetic-tier = plumbing correctness, not field quality ? §6)")
        return 0 if failures == 0 else 1


    def main(argv: Optional[Sequence[str]] = None) -> int:
        ap = argparse.ArgumentParser(
            description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay harness.")
        g = ap.add_mutually_exclusive_group(required=True)
        g.add_argument("--freeze-synthetic", action="store_true",
                       help="generate synthetic trajectory+labels for all built-in scenarios, then "
                            "freeze expected.json+provenance.json + write manifest.json")
        g.add_argument("--refresh-snapshot", action="store_true",
                       help="re-freeze expected.json from COMMITTED trajectory+labels (no regeneration)")
        g.add_argument("--check", action="store_true",
                       help="replay all fixtures + parse-compare vs committed expected.json (exit 0/1)")
        g.add_argument("--freeze-real", action="store_true",
                       help="(P1) de-attribute an OWNED real trajectory+labels into a committable Tier-0 "
                            "fixture; REFUSED without --owned --minors-free and a non-blank --license")
        ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate slug for "
                                        "--freeze-real (default: a random fx_<hex> id)")
        ap.add_argument("--all", action="store_true", help="all slugs for --refresh-snapshot")
        ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the diff!)")
        # --freeze-real (P1 / M2 refusal gate) options:
        ap.add_argument("--trajectory", help="(--freeze-real) source trajectory CSV (Frame,Visibility,X,Y)")
        ap.add_argument("--labels", help="(--freeze-real) source golden labels CSV (start,end)")
        ap.add_argument("--license", dest="license_", default=None,
                        help="(--freeze-real) the source license tag (required, non-blank)")
        ap.add_argument("--owned", action="store_true",
                        help="(--freeze-real) assert owner-recorded Fork-A source (required)")
        ap.add_argument("--minors-free", dest="minors_free", action="store_true",
                        help="(--freeze-real) assert the source contains NO minors (required)")
        ap.add_argument("--fps", type=float, help="(--freeze-real) source fps")
        ap.add_argument("--frame-width", dest="frame_width", type=float, help="(--freeze-real) source frame width (px)")
        ap.add_argument("--label", default="cleared_real", help="(--freeze-real) non-identifying scenario label")
        ap.add_argument("--source-note", dest="source_note", default="",
                        help="(--freeze-real) non-identifying note (NEVER a path/name/id)")
        ap.add_argument("--no-time-origin-reset", dest="time_origin_reset", action="store_false",
                        help="(--freeze-real) DISABLE the anti-attribution time-origin reset (default ON)")
        ap.set_defaults(time_origin_reset=True)
        ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
                        help="override the fixtures directory (default: repo eval_baselines/fixtures)")
        args = ap.parse_args(list(argv) if argv is not None else None)
```

```
    fixtures_dir: Path = args.fixtures_dir

    if args.freeze_synthetic:
        slugs = freeze_synthetic(fixtures_dir)
        print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {', '.join(slugs)}")
        print(f"  -> {fixtures_dir}")
        return 0

    if args.refresh_snapshot:
        if not args.slug and not args.all:
            ap.error("--refresh-snapshot requires --slug S or --all")
        if not args.reason:
            print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can "
                  "rubber-stamp a real regression ? review the visible diff (§6).", file=sys.stderr)
        targets = refresh_snapshot(slug=args.slug, all_slugs=args.all, fixtures_dir=fixtures_dir)
        print(f"[golden-fixtures] refreshed snapshot for: {', '.join(targets)}"
              + (f"  (reason: {args.reason})" if args.reason else ""))
        return 0

    if args.freeze_real:
        missing = [n for n, v in (("--trajectory", args.trajectory),
                                  ("--labels", args.labels),
                                  ("--fps", args.fps),
                                  ("--frame-width", args.frame_width)) if v is None]
        if missing:
            ap.error("--freeze-real requires " + ", ".join(missing))
        try:
            expected = freeze_real_fixture(
                args.label, args.trajectory, args.labels,
                license=args.license_ if args.license_ is not None else "",
                minors_free=args.minors_free, owned=args.owned,
                fps=args.fps, frame_width=args.frame_width,
                fixtures_dir=fixtures_dir, slug=args.slug,
                time_origin_reset=args.time_origin_reset, source_note=args.source_note)
        except RefusalError as e:
            # The code-enforced provenance gate (§4c / §7 ?): refuse loudly, exit non-zero.
            print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}", file=sys.stderr)
            return 2
        print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}' "
              f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
        print("  §6: behaviour-locking, NOT correctness; non-attributable only relative to the "
              "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off (P2).")
        return 0

    # --check
    return _cmd_check(fixtures_dir)


if __name__ == "__main__":
    raise SystemExit(main())
```

5. assistant (2026-06-16T06:27:09.008Z)

Now let me read the test files and the core dependencies the replay reuses.

6. user (2026-06-16T06:27:09.879Z)

```
"""Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic Tier-0.

Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate is
**parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the replay
path), and asserts the Tier-0 privacy invariant positively (no person/audio/pose/gait/face keys).

§6 honest-limits caveat: green here proves only that the deterministic post-trajectory transforms
are unchanged for the frozen synthetic cases ? characterization is behaviour-locking, NOT
correctness, and synthetic-tier numbers measure plumbing, not field quality.
"""
import json

import pytest

from backend.eval import golden_fixtures as gf

# Slugs discovered from the committed manifest ? the suite grows by accretion automatically.
MANIFEST = gf.load_manifest()
SLUGS = [entry["slug"] for entry in MANIFEST]


def test_manifest_is_non_empty():
    """A committed manifest with at least one fixture must exist (else nothing is gated)."""
    assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-synthetic"


@pytest.mark.parametrize("slug", SLUGS)
def test_replay_matches_committed_expected(slug):
    """Replay the fixture and parse-compare against its committed expected.json.

    Characterization = behaviour-locking, NOT correctness (§6): a non-empty mismatch list means
    the deterministic post-trajectory behaviour CHANGED for this frozen case, not that it is wrong.
    """
    committed = gf.load_fixture(slug)["expected"]
    recomputed = gf.replay_fixture(slug)
    mismatches = gf.compare_expected(recomputed, committed)
    assert mismatches == [], (
        f"{slug}: behaviour changed vs frozen snapshot (characterization, not correctness ? §6):\n"
        + "\n".join(f"  - {m}" for m in mismatches)
    )


@pytest.mark.parametrize("slug", SLUGS)
def test_schema_version_le_current(slug):
    """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-compatible)."""
    fx = gf.load_fixture(slug)
    assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
    assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA


```

```
@pytest.mark.parametrize("slug", SLUGS)
def test_no_person_or_audio_keys(slug):
    """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key appears
    anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
    fdir = gf.FIXTURES_DIR / slug
    raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
    for forbidden in gf.FORBIDDEN_KEYS:
        assert forbidden not in raw, (
            f"{slug}: forbidden key substring {forbidden!r} present in expected.json (Tier-0 leak)")

    expected = gf.load_fixture(slug)["expected"]
    for i, cand in enumerate(expected["candidates"]):
        # The only feature block is the shuttle block, and it holds EXACTLY the 5 shuttle cols.
        assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}] has extra keys"
        assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
            f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")


@pytest.mark.parametrize("slug", SLUGS)
def test_synthetic_provenance_tags(slug):
    """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint 5)."""
    prov = gf.load_fixture(slug)["provenance"]
    assert prov["kind"] == "synthetic"
    assert prov["license"] == "synthetic"
    assert prov["minors_free"] is True
    # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
    assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}


def test_control_is_not_learned():
    """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression ? D5)."""
    from backend.eval.experiment import CONTROL
    assert CONTROL.is_learned() is False


def test_perturbing_trajectory_changes_segments(tmp_path):
    """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving the
    windows are RE-DERIVED from the CSV each run, not merely echoed from expected.json."""
    slug = SLUGS[0]
    src = gf.FIXTURES_DIR / slug
    work = tmp_path / slug
    work.mkdir()
    # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived window's
    # `end` shrinks ? a guaranteed structural change in the re-derived segments. (Deleting an
    # interior hole would not move start/end, since merge_gap coalesces across it and the frame
    # indices of the surviving rows are unchanged.)
    lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
    header, body = lines[0], lines[1:]
    mutated = [header] + body[: len(body) // 2]
    (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8", newline="\n")
    (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
                                     encoding="utf-8", newline="\n")
    (work / "provenance.json").write_text((src /
```

```
"provenance.json").read_text(encoding="utf-8"),
                                          encoding="utf-8", newline="\n")

    original = gf.load_fixture(slug)["expected"]
    perturbed = gf.replay_fixture(work)
    assert perturbed["control_segments"] != original["control_segments"], (
        "perturbed trajectory produced identical segments ? replay is echoing, not re-deriving")


def test_freeze_is_idempotent(tmp_path):
    """Re-freezing every built-in fixture into a temp dir produces expected.json structurally
    equal to the committed one (correct-by-construction + deterministic)."""
    slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
    assert slugs, "freeze_synthetic produced no fixtures"
    for slug in slugs:
        committed = gf.load_fixture(slug)["expected"]
        refrozen = json.loads((tmp_path / slug / "expected.json").read_text(encoding="utf-8"))
        # Structural (parsed) equality ? no float churn, exact same shape/values.
        assert gf.compare_expected(refrozen, committed) == [], (
            f"{slug}: re-frozen expected.json differs from committed (non-idempotent freeze)")
        assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally identical"


def test_check_cli_passes(capsys):
    """The --check CLI replays all fixtures and exits 0 when they match the committed snapshots."""
    rc = gf.main(["--check"])
    assert rc == 0

```

7. user (2026-06-16T06:27:10.310Z)

```
"""Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal gate.

These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns an OWNED,
minors-free real trajectory + golden labels into a committable, de-attributed Tier-0 fixture) and
its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.

Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network, NO GPU.
We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory`` and the
``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source" CSVs, then
prove:
  * the refusal gate blocks every unsafe combination (M2);
  * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no FORBIDDEN_KEYS);
  * the time-origin reset is metric-INVARIANT (score identical with/without; only absolute
    ``control_segments`` shift toward 0);
  * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).

§6 honest-limits caveat: a green suite proves only that the deterministic post-trajectory transforms
are unchanged for the frozen cases ? characterization is behaviour-locking, NOT correctness.
```

```python
"""
import pytest

from backend.eval import golden_fixtures as gf


# ---------------------------------------------------------------------------------
# Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
# ---------------------------------------------------------------------------------


def _write_source(tmp_path, *, frame_offset: int = 0):
    """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.

    Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
    ``frame_offset`` frames (and the matching label seconds) to model an arbitrary absolute
    capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
    """
    spec = gf.builtin_specs()[0]  # clean_single_rally
    fps, fw = spec.fps, spec.frame_width
    rows = gf.make_synthetic_trajectory(spec)
    if frame_offset:
        rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
    gts = list(spec.gts)
    if frame_offset:
        off_sec = frame_offset / fps
        gts = [(s + off_sec, e + off_sec) for (s, e) in gts]

    traj = tmp_path / "src_trajectory.csv"
    labels = tmp_path / "src_labels.csv"
    gf.write_trajectory_csv(rows, traj)
    gf.write_labels_csv(gts, labels)
    return traj, labels, fps, fw, gts


# ---------------------------------------------------------------------------------
# M2 ? refusal gate
# ---------------------------------------------------------------------------------


@pytest.mark.parametrize(
    "kwargs, why",
    [
        ({"minors_free": False, "owned": True, "license": "owned"}, "minors_free=False"),
        ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
        ({"minors_free": True, "owned": True, "license": ""}, "license=''"),
        ({"minors_free": True, "owned": True, "license": "   "}, "license=blank"),
        ({"minors_free": True, "owned": True, "license": None}, "license=None"),
    ],
)
def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
    """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-blank license.

    The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-fixture path
    must be impossible to reach silently.
    """
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    with pytest.raises(ValueError) as ei:
        gf.freeze_real_fixture(
            "scenario", traj, labels,
            fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
```

```
    # RefusalError is a ValueError subclass; the message must name a reason.
    assert isinstance(ei.value, gf.RefusalError), why
    assert "REFUSED" in str(ei.value), why
    # Nothing was written for a refused freeze.
    assert not (tmp_path / "fixtures").exists() or not any((tmp_path / "fixtures").iterdir()), (
        f"{why}: a refused freeze must not write any fixture")


# -----------------------------------------------------------------------------------------
# P1 ? de-attribution
# -----------------------------------------------------------------------------------------


def test_real_fixture_is_deattributed(tmp_path):
    """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed surrogate slug,
    no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    expected = gf.freeze_real_fixture(
        "clean_rally", traj, labels,
        license="owned", minors_free=True, owned=True,
        fps=fps, frame_width=fw, fixtures_dir=fixtures)

    slug = expected["slug"]
    assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got {slug!r}"

    prov = gf._read_json(fixtures / slug / "provenance.json")
    assert prov["kind"] == "cleared_real"
    assert prov["minors_free"] is True and prov["owned"] is True
    assert prov["license"] == "owned"
    assert prov["slug"] == slug
    # No attribution keys / no source filename anywhere in the provenance.
    for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
        assert banned not in prov, f"provenance must not carry an attribution key {banned!r}"
    prov_text = (fixtures / slug / "provenance.json").read_text(encoding="utf-8").lower()
    for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
        assert tok not in prov_text, f"source token {tok!r} leaked into provenance"

    # expected.json carries no biometric/identity/non-shuttle stream.
    exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
    for forbidden in gf.FORBIDDEN_KEYS:
        assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in expected.json"

    # The fixture is registered in the manifest with kind=cleared_real.
    manifest = gf.load_manifest(fixtures)
    rows = [e for e in manifest if e["slug"] == slug]
    assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"


def test_supplied_slug_is_honored(tmp_path):
    """An explicitly supplied slug is used verbatim (the surrogate map is the caller's concern)."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    expected = gf.freeze_real_fixture(
        "scenario", traj, labels, slug="fx_custom00",
        license="owned", minors_free=True, owned=True,
        fps=fps, frame_width=fw, fixtures_dir=fixtures)
    assert expected["slug"] == "fx_custom00"
```

```
    assert (fixtures / "fx_custom00" / "expected.json").is_file()


def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
    """Freezing a second real fixture must APPEND to the manifest, not wipe the first."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
                                license="owned", minors_free=True, owned=True,
                                fps=fps, frame_width=fw, fixtures_dir=fixtures)
    e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
                                license="owned", minors_free=True, owned=True,
                                fps=fps, frame_width=fw, fixtures_dir=fixtures)
    slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
    assert {e1["slug"], e2["slug"]} <= slugs


def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
    """source_note is non-identifying by contract ? a path/name in it trips the positive scan."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    with pytest.raises(gf.RefusalError):
        gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_leak0001",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=fixtures,
            source_note=str(traj))  # leaking the source path ? must be refused


# ---------------------------------------------------------------------------------------
# P1 (d) ? time-origin reset is metric-invariant
# ---------------------------------------------------------------------------------------


def test_time_origin_reset_is_metric_invariant(tmp_path):
    """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source frozen
    WITHOUT reset ? but its absolute control_segments shift toward 0 (severed timeline)."""
    offset_frames = 9000  # +5 minutes at 30fps ? a large absolute capture-timeline offset
    traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)

    with_reset = gf.freeze_real_fixture(
        "scenario", traj, labels, slug="fx_reset_on",
        license="owned", minors_free=True, owned=True,
        fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
        time_origin_reset=True)
    without_reset = gf.freeze_real_fixture(
        "scenario", traj, labels, slug="fx_reset_off",
        license="owned", minors_free=True, owned=True,
        fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
        time_origin_reset=False)

    # (1) Score is bit-identical ? metrics.score is built from pure differences.
    assert with_reset["score"] == without_reset["score"], (
        "time-origin reset must be metric-invariant (score built from pure differences)")

    # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
    seg_on = with_reset["control_segments"]
    seg_off = without_reset["control_segments"]
    assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
```

```
    assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start toward 0"
    # The shift equals the offset in seconds (within rounding).
    shift = seg_off[0][0] - seg_on[0][0]
    assert abs(shift - offset_frames / fps) < 1e-3, (
        f"segment shift {shift} should equal the frame offset in seconds {offset_frames / fps}")
    # Reset trajectory starts at frame 0.
    first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
                      .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
    assert first_frame == 0, "with reset, the trajectory must start at Frame 0"


# ---------------------------------------------------------------------------------
# P1 (e/f) ? round-trip / correct-by-construction
# ---------------------------------------------------------------------------------


def test_freeze_real_round_trips_check(tmp_path):
    """After freeze_real_fixture, replay_fixture + compare_expected vs the written expected.json
    returns [] ? idempotent / correct-by-construction (the same path M1's --check uses)."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    expected = gf.freeze_real_fixture(
        "scenario", traj, labels, slug="fx_rt000001",
        license="owned", minors_free=True, owned=True,
        fps=fps, frame_width=fw, fixtures_dir=fixtures)

    fdir = fixtures / expected["slug"]
    committed = gf._read_json(fdir / "expected.json")
    recomputed = gf.replay_fixture(fdir)
    assert gf.compare_expected(recomputed, committed) == [], (
        "a just-frozen real fixture must replay-compare clean (correct-by-construction)")


def test_freeze_real_uses_strict_label_loader(tmp_path):
    """The labels are read through the STRICT gt_loader ? a malformed label row fails loudly,
    never silently shrinks the GT set."""
    fixtures = tmp_path / "fixtures"
    traj, labels, fps, fw, _ = _write_source(tmp_path)
    # Corrupt one label row (end <= start) ? strict loader must raise.
    labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
    with pytest.raises(ValueError):
        gf.freeze_real_fixture(
            "scenario", traj, labels, slug="fx_strict01",
            license="owned", minors_free=True, owned=True,
            fps=fps, frame_width=fw, fixtures_dir=fixtures)

```

8. assistant (2026-06-16T06:27:13.190Z)

Now let me read the core dependencies that the replay reuses ? the algorithm that windowing changes would touch.

9. user (2026-06-16T06:27:14.152Z)

```
"""Distill Gemini (teacher) into a LEARNED local rally classifier (student).

Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
using a few resolution/fps-INVARIANT local features, so the model transfers across
```

```
videos and runs 100% local at inference (no Gemini, privacy lever intact).

Flow (all local at runtime):
  WASB trajectory -> recall-oriented candidate windows (propose many; classifier
  filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
  active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
  -> rally windows.

Training labels come from Gemini full-context labels via the SAME IoU matching the
evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
other videos' candidates, predict the held-out one, score vs its Gemini labels, and
compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.

Run:
    python -m backend.eval.distill_local --manifest videos.json --model-out output/local_rally_model.json
"""
from __future__ import annotations

import json
import argparse
from typing import Dict, List, Optional, Tuple


from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.detectors import rally_gate
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.classifier import LogisticRegression
from backend.eval.metrics import score
from backend.eval.gemini_refine import merge_overlaps
from backend.eval import windowing as _windowing

Interval = Tuple[float, float]

FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]
# Recall-oriented proposal: deliberately permissive so real rallies are among the
# candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
# Single-sourced from backend.eval.windowing so eval and serve can never drift ? see that
# module's docstring for the eval?serve bug these named presets close. The bare tuples are
# preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
# (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
PROPOSAL = _windowing.PROPOSAL.as_tuple()
BASELINE_LOCAL = _windowing.SERVED.as_tuple()   # the served (production) windowing, no learning


def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                     proposal: Tuple[float, float, float] = PROPOSAL) -> Tuple[List[Interval], List[List[float]]]:
    """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = proposal
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    intervals = [(w["start"], w["end"]) for w in wins]
    gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-crossings
    X: List[List[float]] = []
```

```
    for w, g in zip(wins, gated):
        dur = max(1e-6, w["end"] - w["start"])
        win_frames = max(1.0, dur * fps)
        X.append([
            dur,                                              # rally length (sec) ? fps-invariant
            float(w["peak_velocity"]),                        # already normalized by frame_width
            float(w["mean_velocity"]),
            float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ? fps-invariant
            float(g.crossings),                               # net crossings (count) ? the rally signal
        ])
    return intervals, X


def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) -> List[Interval]:
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = BASELINE_LOCAL
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    return [(w["start"], w["end"]) for w in wins]


def load_video(spec: Dict, iou: float = 0.5) -> Dict:
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    intervals, X = build_candidates(spec["trajectory"], fps, fw)
    gts = load_golden_gts(spec["labels"])
    y = label_candidates(intervals, gts, iou)
    return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X": X,
            "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps, fw)}


def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals: List[Interval],
                    threshold: float = 0.5) -> List[Interval]:
    if not intervals:
        return []
    proba = model.predict_proba(X)
    pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
    return merge_overlaps(pos, gap_tol=1.0)


def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
        model_out: Optional[str] = None) -> dict:
    videos = [load_video(s, iou) for s in specs]
    print(f"=== Distilled local rally classifier vs Gemini silver-GT "
          f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
    for v in videos:
        ceil = score(v["intervals"], v["gts"], iou).recall
        print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
              f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

    result: dict = {"videos": [v["name"] for v in videos]}
    if len(videos) >= 2:
        print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-out) ---")
        clf, base = [], []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]
```

```
            X = [row for v in others for row in v["X"]]
            y = [t for v in others for t in v["y"]]
            model = LogisticRegression().fit(X, y, FEATURE_NAMES)
            preds = predict_rallies(model, held["X"], held["intervals"], threshold)
            sc = score(preds, held["gts"], iou)
            bsc = score(held["baseline"], held["gts"], iou)
            clf.append(sc)
            base.append(bsc)
            print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f} R={sc.recall:.3f} F1={sc.f1:.3f}"
                  f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f} F1={bsc.f1:.3f}")
        def mean(xs, a):
            return sum(getattr(s, a) for s in xs) / len(xs)
        print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f} R={mean(clf,'recall'):.3f} "
              f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
              f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
        result["mean_learned_f1"] = mean(clf, "f1")
        result["mean_baseline_f1"] = mean(base, "f1")

    # Final model on ALL videos (for production / running on a fresh held-out video).
    Xall = [row for v in videos for row in v["X"]]
    yall = [t for v in videos for t in v["y"]]
    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
    assert final.w is not None  # set by fit()
    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
    print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
    if model_out:
        final.save(model_out)
        print(f"final model (all videos) -> {model_out}")
        result["model_out"] = model_out
    return result


def main() -> None:
    ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally classifier.")
    ap.add_argument("--manifest", required=True,
                    help="JSON: [{name, trajectory, fps, frame_width, labels}, ...] (labels = Gemini CSV)")
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability cutoff")
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
        specs = json.load(f)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


if __name__ == "__main__":
    main()

```

10. user (2026-06-16T06:27:14.986Z)

```
"""TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).

This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:

    1. Translate the Windows video path to its WSL-visible form (and optionally
       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
    2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
```

```
    3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
    4. Convert that trajectory into high-motion "action windows" ? the same
       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
       pipeline (AI handoff, DB population) is unchanged.

Nothing here imports TensorFlow; all model code stays in WSL.
"""

import os
import csv
import shlex
import logging
import subprocess
from dataclasses import dataclass
from typing import List, Dict, Any, Optional, Tuple

from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin, wsl_tilde_quote

logger = logging.getLogger(__name__)


@dataclass
class TrackNetConfig:
    """Resolved settings for a TrackNet WSL run (built from config.json -> indexing.tracknet)."""
    distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""                          # WSL path to the .h5/.keras weights (REQUIRED)
    queue_length: int = 5
    stage_in_wsl: bool = True                       # copy video into WSL fs first (perf)
    wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live in WSL
    timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no timeout)
    keep_frames: bool = False                       # retain staged video after success (debug only)
    min_free_gb: float = 0.0                        # warn if WSL free space below this (0 = off)

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
        tn = (idx_cfg or {}).get("tracknet", {}) or {}
        return cls(
            distro=tn.get("wsl_distro", "Ubuntu"),
            repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
            conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
            conda_env=tn.get("conda_env", "TrackNetV4"),
            weights_path=tn.get("weights_path", ""),
            queue_length=int(tn.get("queue_length", 5)),
            stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
            wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
            timeout_sec=int(tn.get("timeout_sec", 1800)),
            keep_frames=bool(tn.get("keep_frames", False)),
            min_free_gb=float(tn.get("min_free_gb", 0.0)),
        )


def to_wsl_mnt_path(win_path: str) -> str:
    """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

```

```
    Already-POSIX paths (starting with / or ~) are returned unchanged so the
    same helper is safe to call on either kind of path.
    """
    p = str(win_path)
    if p.startswith("/") or p.startswith("~"):
        return p
    p = p.replace("\\", "/")
    if len(p) >= 2 and p[1] == ":":
        drive = p[0].lower()
        return f"/mnt/{drive}{p[2:]}"
    return p


@dataclass
class TrajectoryPoint:
    frame: int
    visible: bool
    x: float
    y: float


class TrackNetRunner(WslCommandMixin, DetectorRunner):
    """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

    Implements the common DetectorRunner contract so a future Linux-native or
    alternative trajectory runner can be substituted without touching the hybrids.
    """

    def __init__(self, cfg: TrackNetConfig):
        self.cfg = cfg

    def healthcheck(self) -> Tuple[bool, str]:
        """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

        Returns (ok, message). Cheap to call before a long run.
        """
        cmd = (
            f"conda activate {self.cfg.conda_env} && "
            f"python -c \"import tensorflow as tf; "
            f"print('TF', tf.__version__, 'GPUs', len(tf.config.list_physical_devices('GPU')))\""
        )
        try:
            res = self._wsl_bash(cmd)
        except FileNotFoundError:
            return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
        except subprocess.TimeoutExpired:
            return False, "WSL healthcheck timed out."
        out = (res.stdout or "").strip()
        if res.returncode != 0:
            return False, f"WSL env check failed: {(res.stderr or out).strip()}"
        return True, out

    # ----------------------------------------------------------------- inference

    def run_predict(self, video_win_path: str, output_win_dir: str,
                    video_id: Optional[str] = None) -> Optional[str]:
        """Run predict.py on a video. Returns the Windows path to the produced CSV, or None.

        ``output_win_dir`` is a Windows directory (under the repo's output/) that
        WSL writes into via its /mnt view, so the Windows process can read the CSV
        back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
        therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
```

```
        a filename cannot collide on the output CSV.
        """
        if not self.cfg.weights_path:
            logger.error(
                "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                "in config.json to the WSL path of your trained TrackNetV4 weights."
            )
            return None

        # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
        # relative paths through unchanged, so WSL would resolve them against the
        # predict.py cwd instead of the repo (same bug class as wasb_runner).
        output_win_dir = os.path.abspath(output_win_dir)
        os.makedirs(output_win_dir, exist_ok=True)
        # Normalize separators so basename/splitext work when tests (or collector)
        # pass Windows-style paths on a Linux host.
        video_win_path = str(video_win_path).replace("\\", "/")
        base = os.path.basename(video_win_path)
        stem, ext = os.path.splitext(base)

        # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
        if ext.lower() not in (".mp4", ".avi"):
            logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
            return None

        wsl_out = to_wsl_mnt_path(output_win_dir)
        # Namespace by video_id so two same-filename videos don't collide on the CSV.
        # predict.py derives its output name from the (staged) video stem, so staging
        # under the namespaced name propagates into "<key>_predict.csv".
        key = f"{stem}__{video_id[:12]}" if video_id else stem
        expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

        # Resolve the video path WSL will read.
        if self.cfg.stage_in_wsl:
            staged = self._stage_video(video_win_path, f"{key}{ext}")
            if staged is None:
                return None
            wsl_video = staged
        else:
            # No staging: predict.py reads /mnt directly and writes "<stem>_predict.csv".
            # We cannot rename its output, so fall back to the un-namespaced name.
            wsl_video = to_wsl_mnt_path(video_win_path)
            expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

        if self.cfg.min_free_gb > 0:
            free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
            if free is not None and free < self.cfg.min_free_gb:
                logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                               "consider running tools/cleanup_caches.py.",
                               free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

        cmd = (
            f"conda activate {self.cfg.conda_env} && "
            f"cd {self.cfg.repo_dir}/src && "
            f"python predict.py "
            f"--video_path {wsl_tilde_quote(wsl_video)} "
            f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
            f"--output_dir {wsl_tilde_quote(wsl_out)} "
            f"--queue_length {self.cfg.queue_length}"
        )
        logger.info("Running TrackNet inference on %s ...", base)
        try:
            res = self._wsl_bash(cmd)
        except subprocess.TimeoutExpired:
```

```
            logger.error("TrackNet inference timed out after %ss.", self.cfg.timeout_sec)
            return None

        if res.returncode != 0:
            logger.error("TrackNet predict.py failed:\n%s", (res.stderr or res.stdout)[-2000:])
            return None

        if not os.path.exists(expected_csv_win):
            logger.error("TrackNet finished but expected CSV not found at %s", expected_csv_win)
            return None
        logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

        # Success ? the CSV is the durable output; drop the staged video copy (a pure,
        # regenerable intermediate). On earlier failure we return above without cleaning.
        if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
            logger.info("Cache hygiene: removing staged video for %s "
                        "(set indexing.tracknet.keep_frames=true to retain).", key)
            self._wsl_rm_rf([wsl_video])
        return expected_csv_win

    def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
        """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

        Returns the WSL path to the staged copy, or None on failure.
        """
        src_mnt = to_wsl_mnt_path(video_win_path)
        reason = self.wsl_source_unreadable_reason(src_mnt)
        if reason:
            logger.error("Cannot stage video into WSL: %s", reason)
            return None
        dest = f"{self.cfg.wsl_stage_dir}/{base}"
        cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} {wsl_tilde_quote(dest)} && echo OK"
        try:
            res = self._wsl_bash(cmd)
        except subprocess.TimeoutExpired:
            logger.error("Staging video into WSL timed out.")
            return None
        if res.returncode != 0 or "OK" not in (res.stdout or ""):
            logger.error("Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip())
            return None
        return dest

    # --------------------------------------------------------------- CSV -> windows

    @staticmethod
    def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
        """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
        points: List[TrajectoryPoint] = []
        with open(csv_win_path, newline="") as f:
            reader = csv.DictReader(f)
            for row in reader:
                try:
                    points.append(TrajectoryPoint(
                        frame=int(row["Frame"]),
                        visible=int(row["Visibility"]) == 1,
                        x=float(row["X"]),
                        y=float(row["Y"]),
                    ))
                except (KeyError, ValueError):
```

```
                    continue
        points.sort(key=lambda p: p.frame)
        return points

    @staticmethod
    def trajectory_to_action_windows(
        points: List[TrajectoryPoint],
        fps: float,
        frame_width: float,
        chunk_start: float = 0.0,
        velocity_thresh: float = 0.02,
        merge_gap: float = 2.0,
        min_window_duration: float = 2.0,
        chunk_index: Optional[int] = None,
        max_window_duration: float = 0.0,
        min_split_gap: float = 0.5,
    ) -> List[Dict[str, Any]]:
        """Convert a shuttle trajectory into candidate rally windows.

        A frame is "active" when the shuttle is visible AND its inter-frame
        displacement (normalised by frame width, per second) exceeds
        ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
        ``merge_gap`` seconds of each other are grouped into one window; short
        windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
        shape feeding the AI-handoff phase is identical.

        ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard against
        the *over-merge* failure where "shuttle moving" is conflated with "rally in progress"
        and a single window swallows several rallies + the dead time between them. When set, any
        window longer than this is recursively SPLIT at its largest internal inactivity gap (the
        longest stretch with no in-flight shuttle ? the most likely rally boundary), provided
        that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix from
        ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so recall cannot
        drop, and it is config-gated default-OFF until measured on the golden set.
        """
        if fps <= 0:
            fps = 30.0
        if frame_width <= 0:
            frame_width = 1280.0

        # Compute per-frame normalised velocity from the last *visible* point.
        active = []  # (frame_idx, velocity)
        prev: Optional[TrajectoryPoint] = None
        for p in points:
            if not p.visible:
                prev = None  # break the trajectory; absent shuttle = not in flight
                continue
            if prev is not None:
                dframes = p.frame - prev.frame
                if dframes > 0:
                    dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                    velocity = (dist / frame_width) * (fps / dframes)
                    if velocity > velocity_thresh:
                        active.append((p.frame, velocity))
            prev = p

        if not active:
            return []
```

```
        max_gap_frames = int(merge_gap * fps)

        def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
            vels = [v for _, v in group]
            return {
                "start": group[0][0] / fps + chunk_start,
                "end": group[-1][0] / fps + chunk_start,
                "active_frame_count": len(group),
                "peak_velocity": max(vels),
                "mean_velocity": sum(vels) / len(vels),
                "track_id_count": 1,  # single shuttle; kept for window-shape parity
                "chunk_index": chunk_index,
            }

        groups: List[List[Tuple[int, float]]] = []
        group = [active[0]]
        for af in active[1:]:
            if af[0] - group[-1][0] <= max_gap_frames:
                group.append(af)
            else:
                groups.append(group)
                group = [af]
        groups.append(group)

        if max_window_duration > 0:
            groups = TrackNetRunner._split_overlong_groups(
                groups, int(max_window_duration * fps), int(min_split_gap * fps))

        windows = [_finalize(g) for g in groups]
        return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    @staticmethod
    def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames: int,
                               min_split_frames: int) -> List[List[Tuple[int, float]]]:
        """Recursively split any active-frame group longer than ``max_win_frames`` at its largest
        internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most likely rally
        boundary. Splitting only subdivides existing groups (never merges/extends), so it cannot
        invent or drop coverage; a group with no gap ? the floor is left intact (a genuinely long
        rally). Iterative worklist to bound stack depth."""
        if max_win_frames <= 0:
            return groups
        out: List[List[Tuple[int, float]]] = []
        work = list(groups)
        while work:
            g = work.pop()
            if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
                out.append(g)
                continue
            best_i, best_gap = -1, -1
            for i in range(1, len(g)):
                gap = g[i][0] - g[i - 1][0]
                if gap > best_gap:
                    best_gap, best_i = gap, i
            if best_i < 0 or best_gap < min_split_frames:
                out.append(g)  # too long but no real dead-time gap ? keep as one rally
                continue
            work.append(g[:best_i])
            work.append(g[best_i:])
        out.sort(key=lambda grp: grp[0][0])
```

```
        return out

```

11. assistant (2026-06-16T06:27:18.179Z)

Now let me read the rally_gate, experiment, metrics, and gt_loader files.

12. user (2026-06-16T06:27:19.613Z)

```
"""Rally-start gate: suppress between-point false fires using shuttle net-crossings.

WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
the opponent for service after losing a point, that single trajectory looks like a
rally. The discriminator is structural, not duration-based: a real rally has the
shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
single service feed crosses ~once.

This module is camera-agnostic and needs NO new model ? it works on the trajectory
we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
axis with the larger shuttle spread) and the net position (median shuttle coord on
that axis ? the shuttle clusters at the two ends and crosses the net region often,
so the median sits near the net), then counts sign changes of (coord - net) across
the visible points inside a candidate window, with a deadband to ignore jitter.

Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
windows from trajectory_to_action_windows, or to fold into it later.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Optional, Sequence, Tuple

Interval = Tuple[float, float]


@dataclass
class GatedWindow:
    start: float
    end: float
    crossings: int
    visible_points: int
    kept: bool


def _spread(vals: Sequence[float]) -> float:
    """Robust spread = p95 - p5 (ignores a few outlier detections)."""
    if len(vals) < 2:
        return 0.0
    s = sorted(vals)
    lo = s[max(0, int(0.05 * (len(s) - 1)))]
    hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
    return hi - lo


def _median(vals: Sequence[float]) -> float:
    if not vals:
        return 0.0
    s = sorted(vals)
    n = len(s)
    return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


def estimate_play_axis(points) -> Tuple[str, float, float]:
    """Return (axis 'x'|'y', net_value, span) from visible points.
```

```
    Play axis = the image axis along which the shuttle travels most (larger spread).
    For a baseline camera that's Y (players top/bottom); for a side camera, X.
    """
    xs = [p.x for p in points if p.visible]
    ys = [p.y for p in points if p.visible]
    sx, sy = _spread(xs), _spread(ys)
    if sx >= sy:
        return "x", _median(xs), sx
    return "y", _median(ys), sy


def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
    """Count sign changes of (coord - net) across visible points, ignoring the
    deadband zone near the net (so jitter at the net doesn't inflate the count)."""
    crossings = 0
    last_side = None
    for p in window_points:
        if not p.visible:
            continue
        v = (p.x if axis == "x" else p.y) - net
        if abs(v) < deadband:
            continue
        side = v > 0
        if last_side is not None and side != last_side:
            crossings += 1
        last_side = side
    return crossings


def _points_in(points, fps: float, start: float, end: float):
    return [p for p in points if start * fps <= p.frame <= end * fps]


def gate_windows(points, windows: List[Interval], fps: float,
                 min_crossings: int = 2, deadband_frac: float = 0.06,
                 estimate: Optional[Tuple[str, float, float]] = None) -> List[GatedWindow]:
    """Annotate each (start,end) window with its shuttle net-crossing count and a
    keep decision (crossings >= min_crossings). Deadband is a fraction of the play
    axis span. Net/axis are estimated once over the whole trajectory (stable).

    `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
    `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't redone
    per window ? same result, avoids the per-window re-estimation."""
    axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
    deadband = deadband_frac * span
    out: List[GatedWindow] = []
    for (s, e) in windows:
        wp = _points_in(points, fps, s, e)
        c = count_net_crossings(wp, axis, net, deadband)
        vis = sum(1 for p in wp if p.visible)
        out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                               kept=c >= min_crossings))
    return out


def apply_gate(points, windows: List[Interval], fps: float,
               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
    """Convenience: return only the windows that pass the crossing gate."""
    return [(g.start, g.end) for g in
            gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
```

13. user (2026-06-16T06:27:20.496Z)

```
"""Experiment harness ? A/B + shadow + side-by-side for rally detection.

This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
no-regression gate, the pinnable classifier artifact, the persisted-candidate
substrate) already exists ? this module composes it. **No new scoring math lives
here**; everything defers to:

  - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
  - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
  - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
  - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
  - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
  - `backend.eval.calibration`    ? `pct_improvement`

The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
(per-frame signals ? run once on the GPU/WSL box, persisted into
`candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
given persisted candidate features it produces `List[Interval]` deterministically,
with no GPU and zero production risk. So most experiments are pure offline
re-scoring of stored data and never touch the served pipeline.

Three modes (`EXPERIMENT_HARNESS.md` §5):
  - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
    held-out loop, but variant-parameterised.
  - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
    variants agree / diverge (A-only, B-only, agreed). The divergences are what a
    human spot-checks (and what two highlight reels would show side by side).
  - the promotion gate stays `run_regression.py` + `regression_baseline.json`
    (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

Pure + offline + unit-tested. The only DB-touching helper is
`load_video_candidates`, which reads persisted candidates via
`Database.query_candidate_segments`.
"""
from __future__ import annotations

import json
import logging
import os
from dataclasses import dataclass, field
from datetime import datetime, timezone
from hashlib import sha1
from typing import Any, Callable, Dict, List, Optional, Sequence

from backend.eval.calibration import pct_improvement
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
from backend.eval.metrics import Interval, match_intervals, score
from backend.eval.regression import gt_hash
from backend.eval.training import label_candidates
from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count ratio
from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue scores

logger = logging.getLogger(__name__)

# Where a comparison run appends one reproducible record. Tracked location (NOT under
# the gitignored output/) so the leaderboard survives a clean checkout, mirroring
```

```
# regression.DEFAULT_BASELINE_PATH.
DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
_METRICS = ("precision", "recall", "f1", "mean_iou")


class ExperimentError(ValueError):
    """A variant cannot be evaluated as configured (e.g. a learned variant asked to
    score an unlabeled video with no pinned model, or a gate referencing an absent
    feature)."""


# ---------------------------------------------------------------------------- Variant


@dataclass
class Variant:
    """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

    A variant turns one video's persisted candidate windows + features into rally
    intervals. Every knob defaults to the **control** behaviour (no gate, no learned
    filter), so a variant that merely *carries* a new, disabled cue is byte-identical
    to control ? the core no-regression guarantee.

    Fields:
      - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
        for the leaderboard and for selecting the served path (the existing
        `segmenter_registry` + `IndexingConfig` do the actual selection in production).
        New cues are recorded here default-OFF.
      - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
        ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
        `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-shuttle
        windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
      - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
        whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
        cue persists that feature).
      - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
        `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
        required for `compare_unlabeled` on a learned variant.
      - ``feature_names`` ? the persisted features the learned filter consumes. When set
        WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
        (honest) ? the recipe, not a pinned artifact.
      - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
        overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
    """

    name: str
    segmenter: str = "wasb_hybrid"
    config_overrides: Dict[str, Any] = field(default_factory=dict)
    cue_flags: Dict[str, bool] = field(default_factory=dict)
    min_crossings: int = 0
    min_players: int = 0
    classifier_model: Optional[str] = None
    feature_names: Optional[List[str]] = None
    threshold: float = 0.5
    merge_gap: float = 1.0
    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the operating
    # point is chosen on the TRAINING fold, never the held-out one ? fixing the first-A/B
    # failure where a fixed 0.5 zeroed out a small-candidate video.
    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
    calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate proba first

```

```
    def is_learned(self) -> bool:
        """True if this variant applies a learned classifier (pinned model or recipe)."""
        return self.classifier_model is not None or bool(self.feature_names)

    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "segmenter": self.segmenter,
            "config_overrides": self.config_overrides,
            "cue_flags": self.cue_flags,
            "min_crossings": self.min_crossings,
            "min_players": self.min_players,
            "classifier_model": self.classifier_model,
            "feature_names": self.feature_names,
            "threshold": self.threshold,
            "merge_gap": self.merge_gap,
            "tune_threshold": self.tune_threshold,
            "calibrate": self.calibrate,
        }

    @classmethod
    def from_dict(cls, d: dict) -> "Variant":
        known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
        return cls(**{k: v for k, v in d.items() if k in known})

    def spec_hash(self) -> str:
        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
        and makes a result reproducible. The pinned **control** variant always hashes the
        same, so a regression is always traceable to a recipe change, never to drift."""
        blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
        return sha1(blob.encode()).hexdigest()[:12]


#: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
#: filter. Always the comparison reference; "rollback" = make this the served variant.
CONTROL = Variant(name="control")


# ----------------------------------------------------------------- persisted candidates


@dataclass
class VideoCandidates:
    """One video's persisted detection, ready for offline re-scoring.

    ``intervals`` are the candidate windows; ``features`` is column-major
    (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
    ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
    with `load_video_candidates`, or directly in tests.
    """

    name: str
    intervals: List[Interval]
    features: Dict[str, List[float]] = field(default_factory=dict)
    gts: Optional[List[Interval]] = None


def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
    """Persisted feature column, defaulting missing/short columns to 0.0 (matches
    `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
    col = vc.features.get(name)
    n = len(vc.intervals)
    if col is None:
```

```
        logger.warning("variant references feature %r absent from %s ? treating as 0.0",
                       name, vc.name)
        return [0.0] * n
    if len(col) != n:
        raise ExperimentError(
            f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
    return [float(x) for x in col]


def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) -> List[List[float]]:
    cols = [_feature_column(vc, name) for name in feature_names]
    return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]


def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
    """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B players)."""
    n = len(vc.intervals)
    mask = [True] * n
    if variant.min_crossings > 0:
        col = _feature_column(vc, "net_crossings")
        mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
    if variant.min_players > 0:
        col = _feature_column(vc, "players_in_court")
        mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
    return mask


def predict(variant: Variant, vc: VideoCandidates,
            model: Optional[LogisticRegression] = None,
            threshold: Optional[float] = None,
            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
    """Variant prediction: gate by persisted features, optionally filter by a learned
    model, then merge. Pure and deterministic.

    ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
    fold; if omitted and the variant pins ``classifier_model``, that frozen model is
    loaded. A learned variant with neither a supplied nor a pinned model raises ? there
    is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
    fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies them.
    """
    mask = _gate_mask(variant, vc)
    if variant.feature_names:
        if model is None:
            if variant.classifier_model is None:
                raise ExperimentError(
                    f"variant {variant.name!r} is learned but has no model to predict "
                    f"with (pass a fitted model or set classifier_model)")
            model = LogisticRegression.load(variant.classifier_model)
        proba = [float(p) for p in model.predict_proba(_feature_matrix(vc, variant.feature_names))]
        if calibrator is not None:
            proba = [calibrator(p) for p in proba]
        thr = variant.threshold if threshold is None else threshold
        mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
    kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
    return merge_overlaps(kept, gap_tol=variant.merge_gap)


def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                        train_videos: Sequence[VideoCandidates], iou: float
                        ) -> "tuple[Optional[Callable[[float], float]], Optional[float]]":
```

```
    """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
    (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
    (use the variant default). Honest: never looks at the held-out video.
    """
    calibrator: Optional[Callable[[float], float]] = None
    if variant.calibrate:
        raw: List[float] = []
        y: List[int] = []
        for vc in train_videos:
            raw.extend(float(p) for p in
                       model.predict_proba(_feature_matrix(vc, variant.feature_names or [])))
            y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
        if variant.calibrate == "platt":
            calibrator = fit_platt(raw, y)
        elif variant.calibrate == "isotonic":
            calibrator = fit_isotonic(raw, y)
        else:
            raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use 'platt'/'isotonic')")
    threshold: Optional[float] = None
    if variant.tune_threshold:
        best_t, best_f1 = variant.threshold, -1.0
        for k in range(1, 20):                       # grid 0.05..0.95 on the train fold's rally F1
            t = round(0.05 * k, 2)
            f1s = [score(predict(variant, vc, model=model, threshold=t, calibrator=calibrator),
                         vc.gts or [], iou).f1 for vc in train_videos]
            mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
            if mean_f1 > best_f1:
                best_f1, best_t = mean_f1, t
        threshold = best_t
    return calibrator, threshold


# ------------------------------------------------------------------- variant scoring


def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) -> LogisticRegression:
    """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via the
    evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
    X: List[List[float]] = []
    y: List[int] = []
    for vc in videos:
        if vc.gts is None:
            raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
        X.extend(_feature_matrix(vc, variant.feature_names or []))
        y.extend(label_candidates(vc.intervals, vc.gts, iou))
    if not X:
        raise ExperimentError("cannot train a learned variant on zero candidates")
    return LogisticRegression().fit(X, y, variant.feature_names)


def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
    """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
    mean aggregate.

    Prediction policy:
      - **static variant** (no learned filter): predict each video directly ? no training,
        so no leakage; per-video scoring is already honest.
```

```
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO ?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from the train fold
                    assert model is not None                       # _train never returns None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m) for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else 0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }
```

```
def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
    """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare mean)."""
    return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in _METRICS}


def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) -> Dict[str, Any]:
    """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU hides)."""
    gts_by_name = {v.name: (v.gts or []) for v in videos}
    overlaps = _segm.DEFAULT_OVERLAPS
    f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
    ratios: List[float] = []
    for name, preds in eval_v.get("predictions", {}).items():
        gts = gts_by_name.get(name, [])
        got = _segm.f1_at_overlaps(preds, gts, overlaps)
        for ov in overlaps:
            f1k[ov].append(got[ov])
        ratios.append(_segm.segment_count_ratio(preds, gts))

    def _mean(xs: List[float]) -> float:
        return sum(xs) / len(xs) if xs else 0.0
    out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in f1k.items()}
    out["segment_count_ratio"] = _mean(ratios)
    return out


def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                   eval_b: Dict[str, Any]) -> Dict[str, Any]:
    """C1 + C4 honest reporting layered over a compare() pair (additive, EXPERIMENT_HARNESS §6).

    ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in order),
    so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real when
    the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
    """
    a_f1 = [p["f1"] for p in eval_a["per_video"]]
    b_f1 = [p["f1"] for p in eval_b["per_video"]]
    return {
        "variant_a_ci": _ci_block(eval_a),
        "variant_b_ci": _ci_block(eval_b),
        "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
        "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                         "variant_b": _segmentation_block(videos, eval_b)},
    }


def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
            iou: float = 0.5, honest: bool = True, honest_stats: bool = True) -> Dict[str, Any]:
    """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the same
    golden corpus and reports the **B ? A** deltas, per video and in aggregate.

    The deltas use the existing `metrics.score` (per video) + `calibration.pct_improvement`;
    learned variants are scored LOVO-honestly. The result is a self-contained record fit for
    `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with it.
```

```
    ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap CIs, a
    paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ? *alongside* the
    existing bare-mean fields (additive: nothing existing changes). Set False for the legacy
    shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md` §13/C1+C4): a
    bare LOVO mean at n?6 is not trustworthy on its own.
    """
    if len(videos) < 1:
        raise ExperimentError("compare needs at least one labeled video")
    eval_a = evaluate_variant(videos, variant_a, iou, honest)
    eval_b = evaluate_variant(videos, variant_b, iou, honest)

    delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
    delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for m in _METRICS}

    by_video_a = {p["video"]: p for p in eval_a["per_video"]}
    per_video_delta = [
        {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
        for p in eval_b["per_video"]
    ]

    # One fingerprint over the whole label set ? detects "the deltas were measured against
    # different labels" the same way regression.gt_hash guards the no-regression baseline.
    all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]

    result: Dict[str, Any] = {
        "iou": iou,
        "label_set_hash": gt_hash(all_gts),
        "num_videos": len(videos),
        "variant_a": eval_a,
        "variant_b": eval_b,
        "delta": delta,
        "delta_pct": delta_pct,
        "per_video_delta": per_video_delta,
    }
    if honest_stats:
        result["honest"] = honest_summary(videos, eval_a, eval_b)
    return result


# --------------------------------------------------------------- SxS on unlabeled video


def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
                      agree_iou: float = 0.5) -> Dict[str, Any]:
    """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

    With no labels there is no lift to measure ? instead this surfaces where the two
    variants **agree vs diverge**, which is exactly what a human reviews (and what two
    highlight reels would show side by side):

      - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
      - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
      - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
        the human / a later golden label adjudicates).

    Both variants must be predictable without labels: a learned variant therefore needs a
```

```
    pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
    `metrics.match_intervals` ? no new matching logic.
    """
    preds_a = predict(variant_a, video)
    preds_b = predict(variant_b, video)
    mr = match_intervals(preds_a, preds_b, agree_iou)

    agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
              for m in mr.matches]
    agree_ious = [m.iou for m in mr.matches]
    a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
    b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

    def _dur(ivs: List[Interval]) -> float:
        return sum(e - s for s, e in ivs)

    return {
        "video": video.name,
        "agree_iou": agree_iou,
        "variant_a": variant_a.name,
        "variant_b": variant_b.name,
        "num_a": len(preds_a),
        "num_b": len(preds_b),
        "num_agreed": len(agreed),
        "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
        "duration_a": _dur(preds_a),
        "duration_b": _dur(preds_b),
        "agreed": agreed,
        "a_only": a_only,
        "b_only": b_only,
    }


# -------------------------------------------------------------------- leaderboard / DB


def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                      timestamp: Optional[str] = None) -> Dict[str, Any]:
    """Append a compact, reproducible record of a `compare()` result to the JSON
    leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
    `regression_baseline.json`; deliberately the size of a baseline file, not a service
    (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
    """
    row: Dict[str, Any] = {
        "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
        "iou": result.get("iou"),
        "label_set_hash": result.get("label_set_hash"),
        "num_videos": result.get("num_videos"),
        "variant_a": {"name": result["variant_a"]["variant"],
                      "spec_hash": result["variant_a"]["spec_hash"],
                      "aggregate": result["variant_a"]["aggregate"]},
        "variant_b": {"name": result["variant_b"]["variant"],
                      "spec_hash": result["variant_b"]["spec_hash"],
                      "aggregate": result["variant_b"]["aggregate"]},
        "delta": result.get("delta"),
    }
    # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
    # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
    honest = result.get("honest") or {}
    if honest.get("paired_delta_f1"):
        row["honest_delta_f1"] = honest["paired_delta_f1"]
    entries: List[Dict[str, Any]] = []
    if os.path.isfile(path):
        with open(path, encoding="utf-8") as f:
```

```
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via `Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's ``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-touching
        function; the comparison itself is pure and offline. Exercised owner-side on a real DB,
        but works against any `Database` (including an in-memory one in tests).
        """
        rows = db.query_candidate_segments(video_id, detector=detector)
        intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
        features: Dict[str, List[float]] = {}
        for name in feature_names:
            if name == "duration":
                features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in rows]
            else:
                features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in rows]
        return VideoCandidates(name=video_id, intervals=intervals, features=features, gts=gts)

```

14. assistant (2026-06-16T06:27:23.452Z)

Now let me read metrics.py, gt_loader.py, regression.py, and the windowing module to understand the SERVED constant and how the literal vs SERVED divergence works.

15. user (2026-06-16T06:27:24.888Z)

```
    """Pure temporal interval-matching metrics for rally evaluation.

    No I/O, no DB, no global state ? every function here is deterministic and unit
    tested. An "interval" is a `(start, end)` tuple of floats in seconds with
    start < end. We score a list of *predicted* rally intervals against a list of
    *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
    """

    from dataclasses import dataclass, field
    from typing import List, Tuple, Optional

    Interval = Tuple[float, float]


    def interval_iou(a: Interval, b: Interval) -> float:
        """Temporal intersection-over-union of two intervals.

        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
```

```
    overlap / union, both measured as durations on the timeline.
    """
    a_start, a_end = a
    b_start, b_end = b
    if a_end <= a_start or b_end <= b_start:
        return 0.0
    inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
    if inter <= 0.0:
        return 0.0
    union = (a_end - a_start) + (b_end - b_start) - inter
    return inter / union if union > 0 else 0.0


@dataclass
class Match:
    """One matched predicted?ground-truth pair."""
    pred_index: int
    gt_index: int
    iou: float


@dataclass
class MatchResult:
    """Outcome of matching predictions to ground truth at a given IoU threshold."""
    matches: List[Match] = field(default_factory=list)
    unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
    unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives (misses)


def match_intervals(preds: List[Interval], gts: List[Interval],
                    iou_threshold: float = 0.5) -> MatchResult:
    """Greedy one-to-one matching of predictions to ground truth by descending IoU.

    Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
    are claimed highest-IoU first, and each prediction/GT can be used once. This
    keeps a single prediction from being credited for two ground-truth rallies
    (and vice versa).
    """
    candidates = []
    for pi, p in enumerate(preds):
        for gi, g in enumerate(gts):
            iou = interval_iou(p, g)
            # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold` would
            # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
            if iou > 0.0 and iou >= iou_threshold:
                candidates.append((iou, pi, gi))
    # Highest IoU first; ties broken by indices for determinism.
    candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

    used_pred, used_gt = set(), set()
    matches: List[Match] = []
    for iou, pi, gi in candidates:
        if pi in used_pred or gi in used_gt:
            continue
        used_pred.add(pi)
        used_gt.add(gi)
        matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

    unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
    unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
    return MatchResult(matches=matches,
                       unmatched_pred_indices=unmatched_pred,
                       unmatched_gt_indices=unmatched_gt)
```

```python


@dataclass
class Score:
    """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
    iou_threshold: float
    num_pred: int
    num_gt: int
    true_positives: int
    false_positives: int
    false_negatives: int
    precision: float
    recall: float
    f1: float
    mean_iou: float            # over matched pairs (0.0 if none)
    mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
    mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

    def as_dict(self) -> dict:
        return self.__dict__.copy()


def score(preds: List[Interval], gts: List[Interval],
          iou_threshold: float = 0.5,
          match_result: Optional[MatchResult] = None) -> Score:
    """Compute precision/recall/F1, mean matched IoU, and boundary errors.

    Pass a precomputed `match_result` to avoid re-matching (e.g. when also
    extracting the failure lists); otherwise matching is done here.
    """
    mr = match_result if match_result is not None else match_intervals(preds, gts, iou_threshold)

    tp = len(mr.matches)
    fp = len(mr.unmatched_pred_indices)
    fn = len(mr.unmatched_gt_indices)

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0 else 0.0

    if mr.matches:
        mean_iou = sum(m.iou for m in mr.matches) / tp
        mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in mr.matches) / tp
        mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in mr.matches) / tp
    else:
        mean_iou = mean_start_err = mean_end_err = 0.0

    return Score(
        iou_threshold=iou_threshold,
        num_pred=len(preds), num_gt=len(gts),
        true_positives=tp, false_positives=fp, false_negatives=fn,
        precision=precision, recall=recall, f1=f1,
        mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
    )


def pr_curve(preds: List[Interval], gts: List[Interval],
             thresholds: Optional[List[float]] = None) -> List[Score]:
    """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).

    Lets you see strict-vs-lenient behaviour rather than a single operating point.
```

```
    """
    if thresholds is None:
        thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
    return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

16. user (2026-06-16T06:27:25.380Z)

```
"""Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008 prereq).

The repo historically carried two incompatible loaders:
- ``annotations.load_annotations``: positional ``start,end`` (the collector's
  ``curate export`` format), raises on bad rows;
- ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
  (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.

A file in one convention fed to the other loader yielded zero/garbage intervals with no
error. This module accepts BOTH conventions (header-mapped, else positional), so every
quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
flows through one reader. Both legacy entry points now delegate here.

Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
- header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
  columns ? rally_number, ending_reason, sport, ? ? are ignored);
- no header: columns 0,1 positionally.
Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS`` (``annotations.parse_timestamp``).
"""
import csv
import logging
from typing import List, Optional, Tuple

from backend.eval.annotations import parse_timestamp

logger = logging.getLogger(__name__)

Interval = Tuple[float, float]

_START_NAMES = ("start", "start_time")
_END_NAMES = ("end", "end_time")


def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
    """If `row` is a header naming start/end columns, return their indices, else None."""
    cells = [(c or "").strip().lower() for c in row]
    start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
    end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
    if start_idx is not None and end_idx is not None:
        return start_idx, end_idx
    return None


def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
    """Load GT rally intervals from either golden-CSV convention.

    strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
    (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
    strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
    skipped ? a silently shrinking GT set flatters precision.

    Returns intervals sorted by start; overlapping GT intervals are logged as a warning
    (they break one-to-one matching and usually indicate a labeling error).
    """
```

```
    intervals: List[Interval] = []
    skipped = 0
    cols: Optional[Tuple[int, int]] = None   # (start_idx, end_idx); None until detected
    saw_header = False

    with open(csv_path, newline="", encoding="utf-8-sig") as f:
        for lineno, row in enumerate(csv.reader(f), start=1):
            if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
                continue
            if cols is None and not saw_header:
                header = _header_columns(row)
                if header is not None:
                    cols = header
                    saw_header = True
                    continue            # consume the header row
                cols = (0, 1)           # headerless: positional start,end
            assert cols is not None     # every branch above assigns it
            si, ei = cols
            try:
                if len(row) <= max(si, ei):
                    raise ValueError(f"expected at least {max(si, ei) + 1} columns, got {row!r}")
                start = parse_timestamp(row[si])
                end = parse_timestamp(row[ei])
                if start < 0:
                    raise ValueError(f"negative start ({start})")
                if start >= end:
                    raise ValueError(f"start ({start}) >= end ({end})")
            except ValueError as e:
                if strict:
                    raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                skipped += 1
                continue
            intervals.append((start, end))

    if skipped:
        logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path, skipped)

    intervals.sort(key=lambda iv: iv[0])
    for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
        if s2 < e1:
            logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
                           "labeling error? Overlaps break one-to-one matching.",
                           csv_path, s1, e1, s2, e2)
    return intervals

```

17. user (2026-06-16T06:27:26.121Z)

```
"""Regression runner ? verify the production tool against golden-set ground truth.

Implements the explicit ask: *"given a video, get a golden set created and verify
its precision and recall against the production tool."* This is deliberately a thin
wrapper over machinery that already exists:

    ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
    predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
    comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
    baseline      ? a per-video snapshot of "known good" numbers [this module]

A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
```

```
"""
import os
import json
import hashlib
import logging
from dataclasses import dataclass, asdict
from typing import Dict, List, Optional

from backend.infrastructure.database import Database
from backend.eval import metrics
from backend.eval.harness import get_predictions
from backend.annotations import annotation_registry

logger = logging.getLogger(__name__)

# Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
# actually load a committed baseline to compare against.
DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
DEFAULT_TOLERANCE = 0.05


def gt_hash(ground_truth: List[metrics.Interval]) -> str:
    """Stable short hash of the GT intervals ? detects a baseline taken against different labels."""
    norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
    return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


@dataclass
class RegressionResult:
    video_id: str
    stage: str
    iou_threshold: float
    precision: float
    recall: float
    f1: float
    # Baseline values compared against (None when no baseline existed yet).
    baseline_precision: Optional[float]
    baseline_recall: Optional[float]
    tolerance: float
    regressed: bool
    reasons: List[str]
    # True only when a baseline existed to compare against. Distinguishes a genuine PASS
    # from "nothing to compare" so the CLI can refuse to silently green-light the latter.
    has_baseline: bool = False
    gt_hash: Optional[str] = None

    def as_dict(self) -> dict:
        return asdict(self)


# ----------------------------------------------------------------- baseline store

def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
    """Load the per-video baseline snapshot file (returns {} if absent)."""
    if not os.path.isfile(path):
        return {}
    with open(path) as f:
        return json.load(f)


def _baseline_key(video_id: str, stage: str) -> str:
    return f"{video_id}::{stage}"
```

```python

def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                  f1: float, path: str = DEFAULT_BASELINE_PATH,
                  iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                  gt_fingerprint: Optional[str] = None) -> None:
    """Snapshot a video/stage's current numbers as the new 'known good' baseline.

    Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
    later run computed under different settings (or against different labels) is detected
    as incommensurable rather than silently compared.
    """
    baselines = load_baselines(path)
    baselines[_baseline_key(video_id, stage)] = {
        "video_id": video_id, "stage": stage,
        "precision": precision, "recall": recall, "f1": f1,
        "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
    }
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "w") as f:
        json.dump(baselines, f, indent=2, sort_keys=True)
    logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)", _baseline_key(video_id, stage), precision, recall)


# ----------------------------------------------------------------- the runner

def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
            stage: str = "final", iou_threshold: float = 0.5,
            detector: Optional[str] = None,
            tolerance: float = DEFAULT_TOLERANCE,
            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
    """Score stored predictions for one video against ground truth and compare to baseline.

    `ground_truth` is a list of (start, end) intervals ? typically obtained from an
    AnnotationProvider (see `regress_with_provider`). A regression is flagged when
    precision OR recall falls more than `tolerance` below the snapshotted baseline.
    If no baseline exists yet, `regressed` is False (nothing to compare to) and the
    caller can choose to snapshot the current numbers via `save_baseline`.
    """
    preds = get_predictions(db, video_id, stage, detector=detector)
    s = metrics.score(preds, ground_truth, iou_threshold)
    fp = gt_hash(ground_truth)

    baselines = load_baselines(baseline_path)
    b = baselines.get(_baseline_key(video_id, stage))

    reasons: List[str] = []
    regressed = False
    has_baseline = b is not None
    base_p = base_r = None
    if b is not None:
        base_p, base_r = b["precision"], b["recall"]
        # Incommensurable comparison guard: a baseline taken at a different IoU/detector or
        # against different labels is not a valid reference ? flag it instead of trusting it.
        b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"), b.get("gt_hash")
        if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
            regressed = True
            reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold} (incommensurable)")
        if b_det is not None and b_det != detector:
```

```
            regressed = True
            reasons.append(f"baseline detector {b_det!r} != run detector {detector!r} (incommensurable)")
        if b_hash is not None and b_hash != fp:
            regressed = True
            reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels changed)")
        if s.precision < base_p - tolerance:
            regressed = True
            reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} - {tolerance}")
        if s.recall < base_r - tolerance:
            regressed = True
            reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} - {tolerance}")
    else:
        reasons.append("no baseline recorded for this video/stage ? nothing to compare")

    return RegressionResult(
        video_id=video_id, stage=stage, iou_threshold=iou_threshold,
        precision=s.precision, recall=s.recall, f1=s.f1,
        baseline_precision=base_p, baseline_recall=base_r,
        tolerance=tolerance, regressed=regressed, reasons=reasons,
        has_baseline=has_baseline, gt_hash=fp,
    )


def regress_with_provider(db: Database, video_id: str, video_ref: str,
                          tier: str = "file", stage: str = "final",
                          iou_threshold: float = 0.5,
                          detector: Optional[str] = None,
                          tolerance: float = DEFAULT_TOLERANCE,
                          baseline_path: str = DEFAULT_BASELINE_PATH,
                          guideline: Optional[str] = None,
                          sport: str = "badminton",
                          provider_kwargs: Optional[dict] = None) -> RegressionResult:
    """End-to-end: obtain ground truth from the `tier` annotation provider, then regress.

    `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
    once those providers land). `video_ref` is whatever that provider expects (a CSV
    path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
    predictions in the DB (the file MD5).
    """
    provider = annotation_registry.get(tier, **(provider_kwargs or {}))
    gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
    return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                   detector=detector, tolerance=tolerance, baseline_path=baseline_path)


def format_result(r: RegressionResult) -> str:
    verdict = "REGRESSED" if r.regressed else "ok"
    head = (f"[{verdict}] {r.video_id} ({r.stage}) "
            f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
    if r.baseline_precision is not None:
        head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f} tol={r.tolerance}"
    return head + "".join(f"\n   - {why}" for why in r.reasons)

```

18. assistant (2026-06-16T06:27:29.563Z)

Now let me read the windowing module (the SERVED constant), the CI workflow, the docs, and the committed fixtures.

19. user (2026-06-16T06:27:30.870Z)

```
"""Single-sourced candidate-windowing presets ? the eval?serve foundation fix.

**The bug this closes.** The offline eval substrate (``distill_local.build_candidates`` ?
``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was built
on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline
(``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02, merge_gap=2.0,
min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set is
measured on candidates **production never serves**, so a "win" need not transfer. This is
exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served set
(``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).

**The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and the
SERVED preset is **single-sourced from the same Pydantic config defaults production reads**
(:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift. Helpers
build candidates under either preset, and :func:`served_windowing_from_config` lifts the
*actual* served windowing out of a live config (overrides included), so an A/B can target the
operating point a given deployment truly serves.

The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
``per_user_gate``). This module is the **what windowing**; that one is the **promotion gate**.

Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until a
caller opts in (``distill_local`` re-points its module-level presets here, byte-identically).
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, List, Mapping, Tuple

from backend.config.models import IndexingConfig

Interval = Tuple[float, float]

__all__ = [
    "WindowingPreset",
    "SERVED",
    "PROPOSAL",
    "PRESETS",
    "served_windowing_from_config",
    "build_windows",
    "windows_to_intervals",
]


@dataclass(frozen=True)
class WindowingPreset:
    """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
```

```
point.

    ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame counts as
    "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one window;
    ``min_window_duration`` ? windows shorter than this are dropped. These are exactly the three
    knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the windowing.
    """

    name: str
    velocity_thresh: float
    merge_gap: float
    min_window_duration: float
    #: 0 = OFF (default). When > 0, windows longer than this are split at their largest internal
    #: inactivity gap ? the bounded-windowing over-merge fix (W1, RALLY_QUALITY_RESEARCH.md §6).
    max_window_duration: float = 0.0

    def as_tuple(self) -> Tuple[float, float, float]:
        """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple shape the
        ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak. (The
        bounded-windowing knob ``max_window_duration`` is passed separately by ``build_windows``.)"""
        return (self.velocity_thresh, self.merge_gap, self.min_window_duration)

    def to_dict(self) -> Dict[str, Any]:
        return {
            "name": self.name,
            "velocity_thresh": self.velocity_thresh,
            "merge_gap": self.merge_gap,
            "min_window_duration": self.min_window_duration,
            "max_window_duration": self.max_window_duration,
        }


def _served_preset_from_defaults() -> WindowingPreset:
    """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.

    ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
      - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ? wasb/fusion/tracknet)
      - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
      - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)

    Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is the whole
    point: change a served default in ``config/models.py`` and the eval SERVED preset follows,
    so the two can never silently diverge again.
    """
    idx = IndexingConfig()  # construct with defaults ? the served operating point
    return WindowingPreset(
        name="served",
        velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet default 0.02
        merge_gap=float(idx.dead_time_merge_gap),
        min_window_duration=float(idx.min_action_window_duration),
        max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
```

```
(default 0=OFF)
        )


    #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A cue is
    #: only promotable if it holds up HERE ? the operating point real users get.
    SERVED: WindowingPreset = _served_preset_from_defaults()

    #: The recall-oriented PROPOSAL windowing the offline substrate is built on (deliberately
    #: permissive: propose many, let the classifier/gate filter). NOT what production serves ? a
    #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
    #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay valid.
    PROPOSAL: WindowingPreset = WindowingPreset(
        name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
    )

    #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
    PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}


    def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
        """The SERVED windowing a *specific* live config serves (overrides honoured).

        Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against an
        arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter ``family``
        (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a served
        default in ``config.json`` ? the module-level :data:`SERVED` is the *default* operating point;
        this is *this config's* operating point. Falls back to the served defaults for any absent key.
        """
        idx = _as_mapping(config, "indexing")
        family_cfg = _as_mapping(idx, family)
        return WindowingPreset(
            name=f"served:{family}",
            velocity_thresh=float(family_cfg.get("velocity_threshold", SERVED.velocity_thresh)),
            merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
            min_window_duration=float(idx.get("min_action_window_duration", SERVED.min_window_duration)),
            max_window_duration=float(idx.get("max_window_duration", SERVED.max_window_duration)),
        )


    def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
        """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config section
        (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-mapping."""
        if obj is None:
            return {}
        if isinstance(obj, Mapping):
            sub = obj.get(key, {})
        elif hasattr(obj, "get"):
            sub = obj.get(key, {})
        else:
            sub = getattr(obj, key, {})
```

```
    return sub if isinstance(sub, Mapping) else {}


def build_windows(points: List[Any], fps: float, frame_width: float,
                  preset: WindowingPreset) -> List[Dict[str, Any]]:
    """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing call site).

    A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a named preset
    instead of three loose floats, so eval candidate-building and the served path reference the
    SAME knobs by name. Imported lazily to keep this module import-light (no detector deps)."""
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    vt, mg, mwd = preset.as_tuple()
    return TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
        max_window_duration=preset.max_window_duration)


def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
    """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate speak."""
    return [(w["start"], w["end"]) for w in windows]

```

20. user (2026-06-16T06:27:31.102Z)

```
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
  # tracked .py, including scripts/ and training/, which the backend-only
  # py_compile smoke sweep does not reach.
  lint:
    name: Ruff lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github

  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
  # per-module quarantine list ? ratchet by removing entries as modules clean
```

```
  # up. Typed core deps are installed so mypy sees their real signatures
  # (without them ignore_missing_imports would Any-stub half the checks).
  typecheck:
    name: Mypy (lenient baseline)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install mypy + typed core deps
        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai numpy
      - name: Mypy
        run: mypy backend training

  # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
  # module-level import error shipping while the suite stays green.
  # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
  # on machines without the GPU stack (test_import_smoke stubs what's absent).
  import-smoke:
    name: Syntax sweep + import smoke (no GPU stack)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install minimal deps
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
      - name: Run import smoke tests
        run: python -m pytest tests/test_import_smoke.py -v

  full-suite:
    name: Full test suite (offline, mocked)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
          cache: pip
      - name: System libs for opencv-python
        run: |
          sudo apt-get update
          sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y libgl1 libglib2.0-0
      - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
        # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
        # need an out-of-band index that PEP 621 can't express. Keep installing
        # them separately here, before the editable install pulls the rest.
        run: python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
      - name: Install package (editable) + dev tooling
        # Editable PEP 621 install: runtime deps + the `dev` group
        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
        run: python -m pip install -e .[dev]
      # obsreport (private, soft dep) is absent here: reporting tests skip via
      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
      # is therefore calibrated against THIS lane's number (local runs with
      # obsreport installed measure a little higher).
      # Floor calibrated 2026-06-11 against this lane's measured 72% (local
      # with obsreport: 73%). Ratchet it UP as coverage grows; never down
      # without an owner decision.
```

```
      - name: Run full suite (with coverage floor)
        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70

  # Cross-OS determinism guard for the committed golden regression fixtures
  # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
  # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
  # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
  # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
  # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
  golden-crossos:
    name: Golden fixtures cross-OS (${{ matrix.os }})
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai numpy
      - name: Golden-fixtures characterization (cross-OS determinism guard)
        run: |
          python -m backend.eval.golden_fixtures --check
          python -m pytest tests/test_golden_fixtures.py -q

```

21. assistant (2026-06-16T06:27:34.553Z)

Now let me read the docs (the claims §4c/§5/§6) and the committed fixtures + manifest.

22. user (2026-06-16T06:27:35.911Z)

```
# Golden Regression Fixtures ? video-free, non-attributable regression suite

> **Status:** `IN PROGRESS` ? owner-gated · **Owner:** Avi · **Created:** 2026-06-16 · **Last updated:** 2026-06-16 · **Legal:** owner **accepted the researched risk** 2026-06-16 (conditions §3 stand; this is the owner's informed acceptance, **not** a retained-counsel opinion)
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE, per archive policy)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
> **Relation to existing docs:** EXTENDS (does not supersede) `docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` / `docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.
> **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ? it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs **[design]** proposals.

---

## 0. TL;DR

Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-segmentation regression suite from **committed structured files**, so refactors and non-quality PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:

1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory seam** and replay it through the *already-shipped* eval stack (`experiment.py` / `metrics.py` / `windowing.py` / `regression.py`), with two auto-discovered gates: a
```

characterization/snapshot gate (catches any behaviour change) and a **quality-floor** gate (catches quality drops).
2. **Privacy/legal filter (SEALED-FIXTURES)** ? only **de-attributed, minimized, owner-source, biometric-free** numbers are ever committed; richer/sensitive streams stay key-gated or out-of-repo. Non-attributability comes from **deletion-at-source**, not encryption.

The convergence that makes this clean: the *privacy* answer (publish shuttle-only, drop person/pose streams) and the *determinism* answer (freeze only the pure-stdlib `CONTROL` path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the **same minimal public tier**. That coincidence is the spine of the design.

Two findings already actionable, independent of the rest:
- ? **Live de-attribution bug** *(verified)*: real player/venue names are committed today in `eval_baselines/heuristic_lovo_n6.json`, `eval_baselines/gemini_wrapper_2026-06-10.json`, and `backend/eval/eval_partial_labels.py` (the last also has a personal `C:/Users/avidu/...` path) ? and in git history. ? **Deliverable P0.**
- ? **Determinism trap** *(verified, corroborated by two independent red-teams)*: the learned variants (`shuttle_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC_ARCH` logistic regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a contributor's machine. ? the cross-machine gate must be **CONTROL/static-only**.

---

1. Problem & goal

The rally-quality measurement is split across two machines by capability (`docs/GOLDEN_DATA_SHARING.md`):
- `backend/eval/fusion_golden.py` **extracts** `*_golden_features.json` ? needs the **video + trajectory CSV + labels + GPU/WSL** (GPU box only).
- `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` **re-score** those JSONs on **pure CPU, no video** ? any machine.

The CPU re-scoring is *already* video-free, but the inputs live in gitignored `output/` / private OneDrive, so the regression tests that depend on them **skip everywhere** (`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`). The committed `eval_baselines/` baselines exist but `regression.py` still scores DB-stored predictions (needs a pipeline run on the video).

Goal: commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.

---

2. Decisions locked

#	Decision	Source	Implication
D1	**Corpus is owner-recorded / owned**	owner answer 2026-06-16	Legal analysis takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-secret/moat + privacy. Strongest non-attributability (no public reference to correlate against).
D2	**Repo private now, may open-source later**	owner answer 2026-06-16	Design for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later.
D3	**Adults, consent not formalized**	owner answer 2026-06-16	**Person/pose/gait streams stay OUT of the public tier** (DPDP/GDPR derived-personal-data risk without consent). Public tier = **shuttle + abstract intervals + non-biometric features only**.
D4	**Freeze seam = post-detector trajectory** *(design)*	workflow-1 synthesis	Widest deterministic surface; reuse the shipped eval stack verbatim; the raw per-frame CSV is **never committed** (only ~5 aggregate, fps/res-invariant shuttle scalars/window).
D5	**Cross-machine gate is CONTROL/static-only** *(design)*	both red-teams [verified cause]	Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with tolerance, never as a cross-machine byte-snapshot.
D6	**Non-attributability = deletion-at-source, encryption = public-only barrier** *(design)*	workflow-2 synthesis	A contributor who runs the tests necessarily reads

plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only stops the no-key public. |
| D7 | **Two tiers** *(design)* | workflow-2 synthesis | **Tier-0** public/no-key (shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus), **skips cleanly when absent**. |

---

3. Legal foundation (researched ? gates on counsel, §8)

Core verdict (settled across IN/US/EU): the derived numbers ? per-frame shuttle (x,y), rally start/end intervals, audio/court/aggregate-person feature values ? **do NOT inherit the source video's copyright.** They are uncopyrightable *facts/measurements*, not authored expression, and not a §101 "derivative work."

Jurisdiction	Copyright of the data	Database right	Contract / ToS	Net
India (primary)	Not copyrightable as facts (*EBC v. D.B. Modak*); but compilations get thin protection on a low "skill/labour" bar (*Burlington*, *OLX v. Padawan*) ? only bites if **lifted from a 3rd-party dataset**, not self-derived	**None** (no sui-generis right) ? a real permissiveness edge	Anti-scraping ToS (untested in court) + **IT Act s.43/s.66** civil/criminal exposure for 3rd-party ingestion	**Owned-source + minors-excluded + no person/pose = clean**
US	Not copyrightable (*Feist*; §102(b); *NBA v. Motorola*)	None (*Feist* killed sweat-of-the-brow)	**Dominant risk.** ToS enforceable where copyright/CFAA fail (*ProCD*, *hiQ* ? injunction + **forced deletion**). Remedy for redistribution is takedown, not nominal damages	Publishable as facts **iff** owned/permissioned source + no barring ToS + no person/biometric/minor stream
EU / UK	Not copyrightable (*Football Dataco v. Yahoo*)	**Genuinely contestable.** *Football Dataco v. Sportradar*: recording pre-existing on-court facts = "**obtaining**", not "creating" ? our "we created it" premise is **overconfident**; harm test = *CV-Online Latvia*	Ryanair v. PR Aviation: ToS can ban extraction/redistribution even with no IP right	Get EU+UK advice before redistributing numeric streams at scale

Provenance is the decisive fork. **Fork A (owner-recorded, unpublished)** ? the only path safe to commit publicly; collapses to a trade-secret *choice* (Indian breach-of-confidence / Contract Act 1872 / IT Act s.72 ? **not** US DTSA) + privacy. **Fork B (broadcast/YouTube/scraped)** ? do **not** commit; the extraction act needs a license or now-**contested** US fair use (*Thomson Reuters v. Ross* 2025; USCO May-2025), and unlawful acquisition independently sinks it (*Bartz v. Anthropic*, ~$1.5B settlement). Caveat: owner footage **shot at a third party's event** (tournament/club) carries ticket/venue/organiser/broadcaster terms ? treat as Fork B until cleared.

Publish-by-stream (privacy):
- ? **Publish-OK:** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT **intervals** (identity-stripped), non-biometric audio/court/aggregate-person features detached from identity.
- ?? **Conditional (effectively never in v1):** person bounding boxes that *persistently single out a player* ? flips to GDPR Art.9 the moment it identifies. This architecture is prone to that flip ? treat per-player tracking as a tripwire.
- ? **Never publish:** pose/skeleton/**gait** (re-identifies at 80?95% even with faces blurred ? *Weiss et al. 2025*), face crops, any named-player?movement mapping. Matches the existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.
- ? **Minors flip everything:** DPDP "child" = under 18; s.9(3) **prohibits behavioural monitoring** (the Rule-12 exemptions don't cover this project). A persistent per-player signature on a minor is, conservatively, prohibited. **Exclude minors** from any persisted stream beyond inanimate shuttle/interval data.

Non-attributability is relative, not absolute. A per-video trajectory is a fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the **Fork-A source stays unpublished** (no public reference to correlate against). Per *EDPS v. SRB* (CJEU C-413/23 P, 2025) it **remains personal data to the holder** who can re-run the detector. So: conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess as the public-video environment grows). Never market it as "anonymous."

Key authorities: Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C. §101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v. Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v. PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023 s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).

---

4. Architecture (design)

4a. Freeze seam + reuse map
Freeze at the **trajectory CSV ? pure-Python replay**. One shared helper `backend/eval/golden_fixtures.py::replay_fixture()` is the *single* code path used by both the freeze tool and both gates:
`parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates ? rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.
Reuses **verbatim**: `metrics.score/interval_iou`, `segmentation_metrics`, `experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED), `regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0), `eval_baselines/` committed-baseline precedent.

4b. Tiered model (SEALED-FIXTURES)
Tier	Where	Key?	Contents	Test behaviour
Tier-0	`eval_baselines/fixtures/` (tracked)	none	opaque `fixture_id`, fps/frame_width (quantized), relative `start`/`end`, **5 shuttle scalars**, label, relative `gts`. **No person/audio/pose.**	**Unconditional** ? runs on every PR/fork; turns today's skipped litmus into a real public gate
Tier-1	OneDrive (default) **or** SOPS-encrypted in-repo	token/age key	full corpus **with** person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus	**Skips cleanly** when key/data absent (generalize `_golden_present()` ? `_tier1_present()`)

A fresh no-key clone is always **green**; coverage never depends on a secret.

4c. Anti-attribution measures (Tier-0)
- **Opaque RANDOM surrogate id** (`os.urandom` + a private surrogate?source map kept off-git) ? *not* the MD5 `video_id`, and *not* HMAC(salt, video_id?name) (red-team: brute-forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source filename).
- **Metadata strip** ? no filename, video_id, match/event/player identity, capture date.
- **Time-origin reset** ? subtract per-fixture `t0` from all start/end/gts. **Provably metric-invariant** (`metrics.interval_iou` and start/end-error are pure differences) ? bit-identical scores, severed absolute timeline.
- **Stream minimization** ? Tier-0 keeps shuttle block only; person/audio excluded; pose/gait/face have **no field in the schema by design**.
- **Raw-trajectory exclusion** ? never commit the per-frame `(Frame,Visibility,X,Y)` CSV.
- **Quantize fps/frame_width** ? red-team: at n=6 the raw values are a camera/venue quasi-identifier.
- **Provenance hard-fail gate** ? the de-attribution pass refuses to emit a public fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.

4d. The two gates
- **Characterization/snapshot** ? replay fixture, **parse-compare** JSON (never byte-compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields, `approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints (incommensurable ? loud fail). Catches any behaviour change even at constant F1. **CONTROL path only** across machines.
- **Quality-floor** ? score vs committed labels; assert per-video + mean F1 ? floor in `eval_baselines/regression_baseline.json` (created here; carries a **windowing fingerprint** so a served-default change is flagged incommensurable, not silently compared); emit `eval_stats` paired-delta-CI + sign test (NUMBERS INFORM); add a **metrics-vs-base-branch** paired delta. Synthetic-tier floors measure **correctness, not field quality** (documented).

4e. Serialization & optional protobuf

JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is optional (O1)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3 `rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`). Protobuf is **encoding, not secrecy** (trivially `--decode_raw`).

4f. Encryption & keys (Tier-1 only)
Default = out-of-repo + token (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS + age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The **HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus, rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.

---

5. Threat model (who's protected vs not)

Actor	Protected against	NOT protected (by design / residual)
TM1 no-key public	map fixture?source/match/player; decrypt Tier-1; confirm a source even holding the candidate file	read Tier-0 numbers + run shuttle-only harness (intended; useless-in-isolation)
TM2 keyed contributor	recover filename/video_id/match/face/pose/gait ? *never written* (survives the analog hole)	read every value the suite reads (unpreventable; not pretended otherwise)
TM3 CI runner	leaked log can't leak attribution it never had	runs the suite (intended); Tier-1 secret never on fork runners
TM4 malicious keyed insider	correlation needs a public reference; Fork-A source is unpublished ? "not reasonably likely"	**residual:** conditional + revocable ? publish/leak the source later and the link can snap back (forward-looking re-test duty)
TM5 re-ID adversary	source unpublished + only ~5 invariant scalars/window + reset timeline ? no back-match	becomes live if a Fork-A source is ever published
TM6 accidental plaintext commit	required CI leak-scan rejects MD5/known-name/non-surrogate/plaintext-SOPS	if the required check is disabled, a leak lands

---

6. Honest limits ? what a green suite does NOT prove

- Green proves only that the **deterministic post-trajectory transforms** are unchanged for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector, `_probe_fps_width`/cv2 frame-seek, the AI/provider handoff, chunking/re-encode, the ffmpeg stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-chain check.
- Characterization = **behaviour-locking, not correctness** ? it freezes current behaviour *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour correct". This caveat must ride in the **test pass/fail message**, not only the docs.
- **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite synthetic F1 in `QUALITY_ITERATIONS.md` / the paper as rally-detection performance (tag `kind: synthetic`).
- Tier-0 public coverage is **shuttle-only**; the person/audio fusion ?F1 and exact Gate-A values need Tier-1 (skipped on no-key clones).
- Data is **not "anonymous"** absolutely (EDPS v. SRB) ? relative, conditional, revocable, forward-looking.
- Encryption gives **no secrecy from a running contributor**; protobuf gives **no secrecy at all**.
- De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-record provenance gate + counsel sign-off.
- `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the visible diff** + a separate, deliberate floor-baseline update. Require a recorded reason.

---

7. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **small PR per row**, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.

ID	Deliverable	Depends on	Counsel-gated?	Status	PR
P0	**Leak remediation + required CI leak-scan.** Scrub real names/paths in `eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`, `backend/eval/eval_partial_labels.py`, `tests/test_cleanup_caches.py` ? opaque surrogate keys + private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a **required status check**; decide on git history (PR #77, accept vs `filter-repo`).	?	No (owned data hygiene)	?	?
M1	**Fixture framework + synthetic Tier-0 + shared replay helper** (also delivered M2-core + M3 ? see below). `backend/eval/golden_fixtures.py` (`replay_fixture`, schema dispatch, `Path(__file__)`-relative) + 3 **tool-generated** synthetic fixtures + manifest + README + `.gitattributes` LF pin + `tests/test_golden_fixtures.py`. CONTROL-only / pure-stdlib; cross-OS verified.	?	No (synthetic)	?	[#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189)
M2	**Freeze/refresh extractor.** Core (`freeze_synthetic` / `refresh_snapshot` / `--check` CLI + idempotence test) shipped in M1 (#189); the `--license`/`--minors-free`/`--owned` **refusal gate** shipped with P1 (this PR).	M1	No	?	[#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) · _this PR_
M3	**Characterization gate** ? `tests/test_golden_fixtures.py`: parse-compared (exact-int / approx-float 1e-6), **CONTROL-only**, positive no-person/audio assertion, perturbation + idempotence. **Shipped in M1 (#189).** (Windowing/label fingerprint guard + row-count cap ? fold into M4.)	M1	No	?	[#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189)
MX	**Cross-OS determinism guard (CI).** Lightweight `golden-crossos` CI job (ubuntu/windows/macos) running `--check` + the fixtures test ? locks the cross-OS guarantee (CONTROL replay is numpy-only, no GPU stack; a *separate fast job*, not full-suite×3).	M1	No	?	[#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191)
M4	**Quality-floor gate + first `regression_baseline.json`.** `tests/test_golden_quality_floor.py`; create `eval_baselines/regression_baseline.json` with **windowing fingerprint**; metrics-vs-base paired-delta; corpus-level (not per-slug) LOVO for runtime.	M1, M2	No	?	?
P1	**De-attribution + minimization** ? `golden_fixtures.freeze_real_fixture()` (extends the M1 module, not a separate script): **opaque random `fx_<hex>` surrogate slug**, metadata strip, metric-invariant **time-origin reset**, strict GT loader, **positive no-person/audio/MD5/source-path assertion**, manifest non-clobber; `--freeze-real` CLI with the **refusal gate** (exit 2 unless owned + minors-free + license). Tooling-only, 12 synthetic tests; **no real data committed**.	M1	No (tooling)	?	_this PR_
P2	**Owner-accepted (researched-risk), 2026-06-16.** Owner accepted the researched IP/privacy analysis (§3) to proceed **under its conditions** (owned/Fork-A source · minors excluded · biometric/person streams excluded · de-attributed). Owner's informed risk acceptance ? **NOT a retained-counsel opinion**; optional future counsel confirmation per §8.	P1	Owner-accepted	?	?
P3	**Tier-0 REAL subset.** Commit de-attributed, minors/biometric-excluded, owned Fork-A real fixtures (unblocked by P2). Hard-gated **in code** by P1's refusal gate (owned + minors-free + license required) ? needs the owner's per-video clearance list.	P1, P2	conditions §3	?	?
O1	*(optional)* **Protobuf + deterministic serialization.** `rally.fixture.v1` proto3, codegen + `protoc`/`git diff --exit-code` guard, canonical writer, `.gitattributes` binary. Only if cross-OS byte-stable snapshots are wanted.	M1	No	?	?
O2	*(optional)* **Tier-1 key-gated path.** `scripts/fetch_golden.py` (OneDrive token, default) and/or SOPS+age; `_tier1_present()`; `RALLY_GOLDEN_DIR` wired; Tier-1 CI job gated on secret, never on fork runners.	M1	No	?	?
DOC	**Finalize this doc + cross-links.** Add `eval_baselines/fixtures/` row to `CODE_MAP §0` + TEST MAP; cross-link `EVALUATION.md` / `EXPERIMENT_HARNESS.md` / `GOLDEN_DATA_SHARING.md` / `QUALITY_ITERATIONS.md`; update `docs/README.md` index + memory `current-state`/handoff.	M1..M4	No	?	?

Must-fix-before-any-public-commit (red-team), folded into the rows above: ? numeric-tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?); ? scrub names in **all** files + history decision (P0); ? leak-scan a **required** check, hooks are convenience only (P0); ? person-optional loader + **positive** no-person assertion, don't trust the silent `0.0` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop fps/frame_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).

Progress log (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS job green)
PR	Deliverable(s)	Suite	Coverage	Notes
[#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189)	M1 + M2-core + M3	746 pass / 7 skip	80.41%	synthetic Tier-0 + CONTROL replay + characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures)
[#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191)	MX + tracker/legal reconcile	full suite green	80.41%	`golden-crossos` CI job green on ubuntu+windows+macos ? cross-OS **enforced**; legal ? owner-accepted
this PR	P1 + M2 refusal gate	759 pass / 7 skip	**80.49%** (+0.08)	`freeze_real_fixture` + refusal gate; **+13 tests**; tooling-only, no real data committed

---

8. Open owner / counsel questions

Counsel questions ? documented basis (the owner accepted the researched risk on 2026-06-16; see P2). These remain the record, and a retained-counsel confirmation is optional/recommended before scaling beyond a small owned, minors-free, de-attributed subset:
1. Does the EU/UK **sui-generis database right** subsist in our self-derived shuttle/rally streams on the *Sportradar* "obtained vs created" reasoning, and does redistributing a de-attributed Tier-0 subset cause *CV-Online* harm?
2. Post *Thomson Reuters v. Ross* + USCO-2025, is **US fair use for the extraction step** defensible if the dataset could compete in a "data/analytics" market ? does publishing only numbers (never footage) matter?
3. India: are anti-scraping ToS enforceable; **IT Act s.43/s.66** exposure for any 3rd-party ingestion; does pending *ANI v. OpenAI* (~Apr 2026) move it? Confirm **DPDP s.9/Rule 10** and whether per-player tracking is prohibited s.9(3) monitoring of a minor.
4. Sign-off on the **final committed Tier-0 subset + provenance log**; is non-attributability-via-private-source a defensible posture to assert given its conditional/revocable nature?

Owner (strategic):
5. Publishing the Fork-A subset **forfeits the trade-secret moat** (Indian breach-of-confidence framing) ? weighed against community/benchmark/paper aims. Your call.
6. **Forward-looking commitment:** do not publish a highlight reel of the *same* source whose fixtures are committed (without re-clearing), and accept a periodic re-assessment as the public-video environment grows.
7. **Scope of first public commit:** synthetic-only (zero provenance risk, recommended) vs a counsel-cleared real subset.
8. **Tier-1 mechanism:** out-of-repo+token (recommended) vs SOPS+age in-repo. And: is protobuf (O1) wanted, or is canonical-JSON enough?

---

9. Definition of done

The project is **DONE** (flip `Status ? DONE`, archive) when:
- [ ] **P0, M1?M4, P1, DOC** are ? and merged.
- [ ] A clean `git clone` on **win + ubuntu + mac** runs `python run_all_tests.py` and the **Tier-0 gates pass green with no key / no video / no GPU / no DB**.
- [ ] The **CI leak-scan is a required status check** and passes (no MD5 / real-name / non-surrogate / plaintext-SOPS in the tree).
- [ ] Characterization + quality-floor gates are **parse-compared, CONTROL-only, windowing-fingerprinted**, and demonstrably catch a seeded behaviour change.
- [ ] `docs/CODE_MAP.md §0` + TEST MAP updated; this doc cross-linked; `NEXT_STEPS.md` + memory `current-state` point here.
- [ ] **Either** the public commit is synthetic-only, **or** (**P2 owner-accepted ? +

P3**) for a real-data commit, under the §3 conditions enforced **in code** by P1's refusal gate.
- [ ] §6 honest-limits caveat rides in the test pass/fail messages, not only the docs.

Optional O1/O2 may remain ? without blocking DONE (they're earned, not assumed).

---

10. References

Internal code anchors (verified): `backend/eval/{regression.py,fusion_golden.py,fusion_compare.py,ablation.py,experiment.py,metrics.py,windowing.py,distill_local.py,gt_loader.py,eval_stats.py}` · `backend/pipeline/detectors/{base.py,tracknet_runner.py,rally_gate.py}` · `backend/utils/hashing.py::compute_video_id` · `eval_baselines/{heuristic_lovo_n6.json,gemini_wrapper_2026-06-10.json,experiment_leaderboard.json}` · `tests/test_ablation.py::test_litmus_reproduces_gate_a` · `.github/workflows/ci.yml`.
Internal docs: `GOLDEN_DATA_SHARING.md` (#175), `EVALUATION.md`, `EXPERIMENT_HARNESS.md`, `QUALITY_ITERATIONS.md`, `MULTI_SIGNAL_FUSION_PLAN.md`, `CODE_MAP.md §0`.
Legal authorities: see §3 (full URLs in the research workflow artifact). *This section is researched general information, not legal advice.*

Changelog
- **2026-06-16** ? Initial draft synthesized from two multi-agent workflows (mechanics design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verified the live name-leak and the BLAS-determinism cause against code. Status: DRAFT, owner-gated.

23. assistant (2026-06-16T06:27:40.530Z)

Now let me look at the committed fixtures, the manifest, the README, and the .gitattributes for LF pinning. Let me also check the gemini_refine merge_overlaps and classifier to understand the CONTROL path's numpy avoidance claim.

24. user (2026-06-16T06:27:42.722Z)

```
total 28
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609 4018 Jun 16 11:54 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 16 11:54 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 occlusion_break
---MANIFEST---
```

```
[
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "clean_single_rally",
      "expected": "clean_single_rally/expected.json",
      "labels": "clean_single_rally/labels.csv",
      "provenance": "clean_single_rally/provenance.json",
      "trajectory": "clean_single_rally/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "clean_single_rally",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
```

```
    }
  },
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "multi_rally_with_deadtime",
      "expected": "multi_rally_with_deadtime/expected.json",
      "labels": "multi_rally_with_deadtime/labels.csv",
      "provenance": "multi_rally_with_deadtime/provenance.json",
      "trajectory": "multi_rally_with_deadtime/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "multi_rally_with_deadtime",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  },
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "occlusion_break",
      "expected": "occlusion_break/expected.json",
      "labels": "occlusion_break/labels.csv",
      "provenance": "occlusion_break/provenance.json",
      "trajectory": "occlusion_break/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "occlusion_break",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  }
]
```


```
---SLUG DIRS---
eval_baselines/fixtures/clean_single_rally/:
expected.json
labels.csv
provenance.json
trajectory.csv

eval_baselines/fixtures/multi_rally_with_deadtime/:
expected.json
labels.csv
provenance.json
trajectory.csv

eval_baselines/fixtures/occlusion_break/:
expected.json
labels.csv
provenance.json
trajectory.csv
```

25. user (2026-06-16T06:27:43.111Z)

```
"""Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
```

```

WASB (shuttle-only) over-fires on between-point service feeds. This drives the
EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
each candidate window is cut to a small padded clip and handed to Gemini, which ?
seeing the actual footage ? returns the true rally inside it (tight boundaries) or
nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
not GB-scale chunks; high-precision because Gemini judges each candidate visually.

Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.

This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.

Run (GEMINI_API_KEY in env):
    python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv \
        --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
"""
from __future__ import annotations

import os
import os.path as osp
import time
import argparse
import concurrent.futures
from typing import List, Tuple, Optional

from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry

Interval = Tuple[float, float]


def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
    """Union of overlapping intervals, optionally stitching across a small gap.

    gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal brief
    shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally gaps
    (several seconds) as separate rallies. (Replaces the old envelope-per-candidate logic,
    which over-merged multiple real rallies WASB had bundled into one candidate.)"""
    merged: List[Interval] = []
    for s, e in sorted(intervals):
        if merged and s <= merged[-1][1] + gap_tol:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return merged


def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
    """Map a Gemini clip's returned segments back to full-video seconds, clamped to clip."""
    out: List[Interval] = []
    for sg in raw or []:
        try:
            st = float(sg["start_time"]) + clip_start
            en = float(sg["end_time"]) + clip_start
```

```
        except (TypeError, ValueError, KeyError):
            continue
        st = max(clip_start, st)
        en = min(clip_end, en)
        if en > st:
            out.append((round(st, 3), round(en, 3)))
    return out


def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
                  pad: float, duration: float, tmpdir: str, idx: int,
                  clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval, List[Interval]]:
    # Own provider/client per call: the genai client isn't reliably thread-safe, and a
    # shared one across workers throws socket errors. A fresh lazy client is cheap.
    provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
    s, e = window
    cs = max(0.0, s - pad)
    ce = min(duration, e + pad) if duration > 0 else e + pad
    clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
    # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
    # 500MB upload cap even for 4K/5K sources.
    if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True, scale_width=clip_scale_width):
        return idx, window, []
    try:
        prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
        raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs, label=f"cand{idx}")
        mapped = _offset_segments(raw, cs, ce)
    finally:
        try:
            os.remove(clip)
        except OSError:
            pass
    # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
    # (merge_gap bridged the between-point gaps), and collapsing them to one envelope swept
    # dead time into a "rally" (precision loss). Tiny invisibility splits are healed later
    # by the gap-tolerant global merge in run().
    return idx, window, mapped


def run(video: str, trajectory: str, fps: float, frame_width: float,
        cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
        max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
        min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) -> dict:
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        raise SystemExit("GEMINI_API_KEY not set in env")
    sport_profile = sport_registry.get("badminton")
    duration = get_video_duration(video)

    candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
    if limit:
        candidates = candidates[:limit]
    tmpdir = osp.join("output", "gemini_refine_clips")
    os.makedirs(tmpdir, exist_ok=True)

    print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini {model}, "
          f"pad {pad}s, {max_workers} workers", flush=True)
    t0 = time.time()
```

```
    per_candidate = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
        futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad, duration,
                          tmpdir, i, clip_scale_width)
                for i, w in enumerate(candidates)]
        for fut in concurrent.futures.as_completed(futs):
            idx, window, mapped = fut.result()
            per_candidate.append((idx, window, mapped))
            tag = "REJECT (no rally)" if not mapped else \
                  " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in mapped)
            print(f"  cand {idx:>2} {seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)

    raw_intervals = [iv for _, _, m in per_candidate for iv in m]
    refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >= min_dur]
    write_segments_csv(out_csv, refined)
    try:
        os.rmdir(tmpdir)
    except OSError:
        pass

    kept_cands = sum(1 for _, _, m in per_candidate if m)
    play = sum(e - s for s, e in refined)
    print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
          f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
          f"({time.time() - t0:.0f}s)", flush=True)
    return {"refined": refined, "num_candidates": len(candidates),
            "candidates_with_rally": kept_cands, "out_csv": out_csv}


def full_video_rallies(video: str, model: str = "gemini-flash-latest",
                       out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width: Optional[int] = 1280,
                       min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float = 10.0,
                       max_workers: int = 3) -> List[Interval]:
    """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under the
    upload cap), so it finds every rally itself ? not bounded by WASB's candidate recall.
    Independent of the trajectory; complements refine()."""
    api_key = os.environ.get("GEMINI_API_KEY")
    if not api_key:
        raise SystemExit("GEMINI_API_KEY not set in env")
    sport_profile = sport_registry.get("badminton")
    duration = get_video_duration(video)
    starts, t = [], 0.0
    while t < duration:
        starts.append(t)
        t += max(1.0, chunk_sec - overlap)
    tmpdir = osp.join("output", "gemini_full_clips")
    os.makedirs(tmpdir, exist_ok=True)

    def do_chunk(ci: int, cstart: float) -> List[Interval]:
        cend = min(cstart + chunk_sec, duration)
        clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
        if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True, scale_width=clip_scale_width):
            return []
        try:
            provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
```

```
            prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
            raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart, label=f"chunk{ci}")
            return _offset_segments(raw, cstart, cend)
        finally:
            try:
                os.remove(clip)
            except OSError:
                pass

    print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width {clip_scale_width}) "
          f"-> Gemini {model}", flush=True)
    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
        chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in enumerate(starts)]]
    rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c], gap_tol=1.0) if e - s >= min_dur]
    write_segments_csv(out_csv, rallies)
    try:
        os.rmdir(tmpdir)
    except OSError:
        pass
    play = sum(e - s for s, e in rallies)
    print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}", flush=True)
    return rallies


def main() -> None:
    ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
    ap.add_argument("--video", required=True)
    ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required unless --full-video)")
    ap.add_argument("--fps", type=float, default=59.94)
    ap.add_argument("--frame-width", type=float, default=1920.0)
    ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
                    help="WASB candidate config to feed Gemini (baseline = whole rallies, good recall)")
    ap.add_argument("--pad", type=float, default=3.0)
    ap.add_argument("--model", default="gemini-flash-latest")
    ap.add_argument("--max-workers", type=int, default=4)
    ap.add_argument("--min-dur", type=float, default=2.0)
    ap.add_argument("--out", default="output/gemini_refined.csv")
    ap.add_argument("--limit", type=int, default=0, help="only the first N candidates (smoke test)")
    ap.add_argument("--clip-scale-width", type=int, default=1280,
                    help="downscale uploaded clips to this width (0=keep source res)")
    ap.add_argument("--full-video", action="store_true",
                    help="full-context pass: send the whole clip (chunked/downscaled) to Gemini "
                         "instead of refining WASB candidates (needs no --trajectory)")
    args = ap.parse_args()
    scale = args.clip_scale_width or None
    if args.full_video:
        full_video_rallies(args.video, model=args.model, out_csv=args.out,
                           clip_scale_width=scale, min_dur=args.min_dur, max_workers=args.max_workers)
        return
    if not args.trajectory:
        raise SystemExit("--trajectory is required unless --full-video")
    cfg = BASELINE if args.config == "baseline" else TUNED
    run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
        model=args.model, max_workers=args.max_workers, out_csv=args.out,
        min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
```

```
if __name__ == "__main__":
    main()
```

26. assistant (2026-06-16T06:27:45.870Z)

Now let me look at the committed expected.json files and the README, plus the .gitattributes for LF pinning and the classifier (to confirm CONTROL avoids numpy import).

27. user (2026-06-16T06:27:49.670Z)


```
=== clean_single_rally/expected.json ===
{
  "candidates": [
    {
      "end": 3.966667,
      "shuttle": {
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 0.033333
    }
  ],
  "control_segments": [
    [
      0.033333,
      3.966667
    ]
  ],
  "gt_hash": "6c93801fd607",
  "iou": 0.5,
  "schema_version": 1,
  "score": {
    "f1": 1.0,
    "false_negatives": 0,
    "false_positives": 0,
    "iou_threshold": 0.5,
    "mean_end_error": 0.033333,
    "mean_iou": 0.983333,
    "mean_start_error": 0.033333,
    "num_gt": 1,
    "num_pred": 1,
    "precision": 1.0,
    "recall": 1.0,
    "true_positives": 1
  },
  "slug": "clean_single_rally"
}
=== provenance ===
{
  "description": "One continuous ~4s rally; shuttle oscillates across the net ? 1 window.",
  "fps": 30.0,
  "frame_width": 1280.0,
  "iou": 0.5,
  "kind": "synthetic",
  "license": "synthetic",
  "minors_free": true,
  "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. Plumbing-
correctness fixture ? NOT a measure of rally-detection quality (§6).",
  "paths": {
```

    "expected": "expected.json",
    "labels": "labels.csv",
    "trajectory": "trajectory.csv"
  },
  "schema_version": 1,
  "slug": "clean_single_rally",
  "windowing": {
    "max_win": 0.0,
    "mg": 2.0,
    "mwd": 2.0,
    "vt": 0.02
  }
}

=== labels ===

start,end
0.000000,4.000000

=== README ===

`eval_baselines/fixtures/` ? Golden Regression Fixtures (Tier-0, synthetic)

Video-free, **CONTROL-only** characterization fixtures for the rally-segmentation pipeline. A contributor on **any machine ? no video, GPU, WSL, or DB** ? replays these committed structured files through the already-shipped eval stack and gets a red/green regression signal. See `docs/GOLDEN_REGRESSION_FIXTURES.md` (deliverable **M1**; §4, §6, §7).

What's here

Each fixture is one directory:

File	Purpose
`trajectory.csv`	Synthetic shuttle trajectory (`Frame,Visibility,X,Y`) ? the freeze seam input.
`labels.csv`	Synthetic ground-truth rally intervals (`start,end`).
`expected.json`	Frozen replay snapshot (candidates + 5 shuttle features + CONTROL segments + score).
`provenance.json`	Pinned windowing params + privacy/licensing tags + description.

`manifest.json` lists every fixture; `backend/eval/golden_fixtures.py` is the shared replay module + freeze/refresh/check CLI.

Guardrails (non-negotiable)

- **Synthetic / owned only.** Every fixture is generated by deterministic formulae (NO random) ? zero third-party footage, zero provenance/licensing risk. Tagged `kind: "synthetic"`, `license: "synthetic"`, `minors_free: true`.
- **No biometric / identity streams.** Tier-0 carries the **5 shuttle feature columns only** (`duration, peak_velocity, mean_velocity, active_ratio, net_crossings`). There is, by design, **no person / pose / gait / face / audio / player field anywhere in the schema** ? a test asserts this positively (deletion-at-source, §4c). Minors and biometrics are excluded.
- **CONTROL / pure-stdlib only.** The replay uses `experiment.CONTROL` (`is_learned() == False`) ? no numpy / logistic-regression. Learned variants' near-threshold decisions flip across CPUs, so they can **never** anchor a cross-machine snapshot (§4d / D5).
- **Parse-compare, never byte-compare.** The gate `json.load`s both sides: exact equality on int/structural fields, abs-tolerance `1e-6` on floats (CRLF/float-format safe).
- **Windowing is pinned per fixture.** The `(vt, mg, mwd, max_win)` params live in each `provenance.json` and are passed explicitly to `build_candidates` ? the replay does **not** depend on the named `windowing.SERVED` constant, so a future SERVED retune can't silently churn fixtures.

Fixtures are GENERATED, never hand-edited

Do **not** hand-edit `expected.json` / `provenance.json` / the CSVs. They are produced correct-by-construction by the CLI.

```bash
# Generate the synthetic trajectory.csv + labels.csv for all built-in scenarios, then freeze expected.json + provenance.json + (re)write manifest.json:
python -m backend.eval.golden_fixtures --freeze-synthetic

# Replay all fixtures and parse-compare vs the committed expected.json (exit 0/1):
python -m backend.eval.golden_fixtures --check

# Re-freeze expected.json from the COMMITTED trajectory+labels (NO regeneration). ? T rubber-stamp a real regression ? review the visible diff and record a reason (§6):
python -m backend.eval.golden_fixtures --refresh-snapshot --all  --reason "<why>"
python -m backend.eval.golden_fixtures --refresh-snapshot --slug clean_single_rally --reason
"<why>"
```

Honest limits (§6 ? rides in the test pass/fail messages, not only here)

```
- A green suite proves only that the **deterministic post-trajectory transforms** are unchanged
  for the frozen cases. It does **not** cover the WASB/TrackNet detector, frame decoding, the
  AI/provider handoff, chunking, the ffmpeg stitch, DB persistence, or the API filter.
- Characterization is **behaviour-locking, not correctness** ? it freezes current behaviour
  *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour correct".
- **Synthetic-tier F1 measures plumbing correctness, NOT field quality.** Never cite a synthetic
  fixture's F1 in `QUALITY_ITERATIONS.md` or the paper as rally-detection performance.
```

28. user (2026-06-16T06:27:51.009Z)

```
=== .gitattributes ===
# Golden Regression Fixtures (M1) ? pin LF line endings so the committed fixtures are byte-stable across OSes despite core.autocrlf. The characterization gate parse-compa (never byte-compares) JSON, but a stable on-disk form keeps `git diff` reviewable and fixtures reproducible. See docs/GOLDEN_REGRESSION_FIXTURES.md §4d/§4e.
eval_baselines/fixtures/** text eol=lf
=== fixtures .gitattributes ===
=== classifier head ===
"""A tiny, dependency-free logistic-regression classifier (numpy only).

The offline learned segmenter (ADR-004) must run locally with no external API and
no heavy ML stack ? the project already depends on numpy, and sklearn is not
installed. This implements just enough: standardized features, balanced-class
gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
model is a small, human-readable, version-controllable file.

It deliberately stays simple: the rally-window features are few and roughly
linearly separable (high player motion + sustained activity => real rally), so a
well-regularized linear model is a sensible, inspectable baseline. Swap in a
richer model later behind the same fit/predict_proba/save/load interface.
"""

import json
from typing import List, Optional

import numpy as np


class LogisticRegression:
    def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
        self.lr = lr
```

```
        self.epochs = epochs
        self.l2 = l2
        self.w: Optional[np.ndarray] = None
        self.b: float = 0.0
        self.mean: Optional[np.ndarray] = None
        self.std: Optional[np.ndarray] = None
        self.feature_names: Optional[List[str]] = None

    @staticmethod
    def _sigmoid(z: np.ndarray) -> np.ndarray:
        # Clip for numerical stability (avoid overflow in exp).
        return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

    def _standardize(self, X: np.ndarray) -> np.ndarray:
        return (X - self.mean) / self.std

    def fit(self, X, y, feature_names: Optional[List[str]] = None,
```

29. assistant (2026-06-16T06:27:58.149Z)

Now I have a comprehensive picture. Let me verify a few critical things: (1) whether `experiment.py` imports numpy at module level (the CONTROL "no numpy execution" claim ? but the import itself happens), (2) the key robustness concern about the `max_win` parameter not being passed through the replay, and (3) what happens with the net-crossings gate (`min_crossings=0`) interaction.

Let me check the critical robustness gap: the replay calls `build_candidates(proposal=proposal)` where `proposal` is a 3-tuple `(vt, mg, mwd)`. The `max_win` (PINNED_MAX_WINDOW) is pinned in provenance but **never passed** to `build_candidates`, because `build_candidates` doesn't accept `max_window_duration`.

30. user (2026-06-16T06:27:59.698Z)

No matches found

31. user (2026-06-16T06:28:00.757Z)

No matches found

32. assistant (2026-06-16T06:28:05.813Z)

Confirmed: `distill_local.build_candidates` does NOT accept or pass `max_window_duration`. This is the core robustness gap I need to analyze in detail. Let me verify the exact signature and what happens if someone enables bounded-windowing (W1, PR #187) via SERVED.

Let me trace the precise control flow for the windowing-change scenarios the lens asks about. Let me also confirm whether `predict(CONTROL, vc)` ignores `net_crossings` entirely (since `min_crossings=0` for CONTROL).

33. user (2026-06-16T06:28:06.783Z)

```
    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                         proposal: Tuple[float, float, float] = PROPOSAL) -> Tuple[List[Interval], List[List[float]]]:
        """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = proposal
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        intervals = [(w["start"], w["end"]) for w in wins]
        gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-crossings
```

```
        X: List[List[float]] = []
        for w, g in zip(wins, gated):
            dur = max(1e-6, w["end"] - w["start"])
            win_frames = max(1.0, dur * fps)
            X.append([
                dur,                                              # rally length (sec) ? fps-invariant
                float(w["peak_velocity"]),                        # already normalized by frame_width
                float(w["mean_velocity"]),
                float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ? fps-invariant
                float(g.crossings),                               # net crossings (count) ? the rally signal
            ])
        return intervals, X
```

34. user (2026-06-16T06:28:11.679Z)

```
=== expected.json for multi+occlusion (net_crossings + control_segments) ===
clean_single_rally [[0.033333, 3.966667]] [7]
multi_rally_with_deadtime [[0.033333, 4.0], [7.033333, 10.966667]] [7, 7]
occlusion_break [[0.033333, 3.966667], [7.033333, 10.966667]] [7, 7]
=== git log for windowing/golden PRs ===
8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate (#192)
64494ef feat(eval): golden regression fixtures M1 ? video-free CONTROL-only characterization harness (#189)
a3b209a feat(windowing): Tier 1 (W1) ? bounded windowing fixes the over-merge (?F1 +0.139, 6/0) (#187)
46ad601 fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL (#177)
4251c9c fix(eval): close the eval?serve windowing gap ? single-source SERVED preset + served-regression promotion guard (#154)
7651687 feat: cache hygiene ? self-cleaning pipeline + cleanup tool + disk guard (#109)
31069cb fix(detectors): keep ~ expandable in WSL staging/inference commands (#97)
2c241a8 refactor: collapse the hybrid-segmenter twins into a shared trajectory base
c64a714 fix(reliability): bound WSL/provider timeouts, SQLite WAL+busy_timeout, chunk-step guard, CORS (#69)
34cdcfa fix(detectors): namespace WSL/cache/output artifacts by video_id, not basename (#64)
49272e5 feat(detector): finish DetectorRunner ABC (complete WSL de-risk seam)
7b9dad2 fix(tests): cross-platform support for Windows-style paths in eval batch resolver and tracknet/wasb basename logic
8615534 refactor: adopt approved layered directory structure
```

35. assistant (2026-06-16T06:28:17.520Z)

This is the central robustness finding. Let me now precisely characterize the divergence. The fixtures pin `max_win=0.0` (W1 bounded-windowing OFF). The SERVED preset sources `max_window_duration` from `IndexingConfig` defaults (currently 0.0). But:

1. The replay path `build_candidates(proposal=(vt,mg,mwd))` is a **3-tuple only** ? it physically **cannot** pass `max_window_duration` to `trajectory_to_action_windows`. So the pinned `max_win` in provenance is **decorative** in the replay ? it is read into manifest/provenance but the replay never consumes it (`_pinned_proposal` returns only 3 values, and `build_candidates` has no `max_win` param).

Let me verify exactly what `_pinned_proposal` does and that `max_win` is never threaded into the replay, and confirm what would happen if W1 were turned on in SERVED defaults.

36. user (2026-06-16T06:28:19.234Z)

```
    def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
        """The windowing proposal tuple, read from the fixture provenance when present (the
```

```
pinned
289         params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
290         if prov and "windowing" in prov:
291             w = prov["windowing"]
292             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
293         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
294
295
296     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
297         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
298
299         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
300         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
301
302             parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
303             cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
304
305         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
306         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
307         merges overlaps; NO numpy is touched.
308         """
309         fdir = _resolve_dir(slug_or_dir)
310         slug = fdir.name
311         traj_csv = fdir / "trajectory.csv"
312         labels_csv = fdir / "labels.csv"
313         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
314
315         fps, fw = _fps_fw(prov, slug)
316         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
317         proposal = _pinned_proposal(prov)
318
319         intervals, feature_rows = distill_local.build_candidates(
320             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
321
```

37. user (2026-06-16T06:28:25.087Z)

```
=== IndexingConfig max_window_duration default ===
138:    max_window_duration: float = 0.0
=== windowing.build_windows callers (does golden use it?) ===
26:  ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn fixtures.
69:#: here on purpose ? the spec forbids depending on the named ``windowing.SERVED`` constant so
a
289:    params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
```

38. assistant (2026-06-16T06:28:33.965Z)

Now I have confirmed the core robustness findings. Let me verify a couple more edge cases to be thorough and honest:

1. The `parse_trajectory_csv` swallows malformed rows silently (`except (KeyError, ValueError): continue`) ? but that's the detector path, and the fixtures are CONTROL synthetic so it's lower risk.
2. The `_scan_forbidden` substring scan for `"player"` ? does `_round`/any field write "player"? And does `interval_iou` mean_iou could be 0.983 (>1.0 active_ratio noted).
3. The W1 `_split_overlong_groups` interaction: if SERVED's `max_window_duration` becomes > 0 in a future PR, the **replay would not change** (good for fixtures) but the production path WOULD ? meaning a fixture passing green tells you NOTHING about whether the W1 change is correct. This

is exactly the "passes silently" risk.

Let me confirm the `build_windows` function (windowing.py) ? the doc §4a claims the replay uses `windowing.build_windows(preset)`, but the actual code uses `distill_local.build_candidates`. Let me verify this doc-vs-code divergence and check whether `min_split_gap` is also unpinned.

39. user (2026-06-16T06:28:38.203Z)

```
=== Does replay touch windowing.build_windows? (doc 4a claims it does) ===
13:    parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
25:  provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the named
302:        parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5 shuttle
319:    intervals, feature_rows = distill_local.build_candidates(

=== min_split_gap default + W1 split knobs ===
265:        max_window_duration: float = 0.0,
266:        min_split_gap: float = 0.5,
277:        ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
282:        that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
from
334:        if max_window_duration > 0:
336:                groups, int(max_window_duration * fps), int(min_split_gap * fps))

=== check: would 'player' substring ever be emitted by writers? scan FEATURE/key names ===
23:  person / pose / gait / face / audio field anywhere in the schema (positive-assertion test).
88:FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
323:    # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
324:    # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
508:    re-introducing a person/audio/pose stream or an id/path back-channel ? never the silent
default.
515:                f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player MUST
NOT "
577:    The committed bytes carry NO filename / video_id / match / player / capture-date ? only
an
```

40. assistant (2026-06-16T06:28:45.642Z)

I now have everything I need. Let me do a final check on two remaining edge cases: the `source_note` leak scan when `time_origin_reset=False` and frame `0` handling, and whether the `_scan_forbidden` source-path scan could be bypassed by a short token. Let me also verify whether the freeze_real path's `frame_width` is recorded but a mismatch with the actual trajectory could go undetected, plus the schema_version downgrade (only `>` is rejected, equal/older accepted).

Let me confirm one determinism subtlety: `_fmt`/`_round` use Python's `round()` (banker's rounding), and floats are written at 6dp but compared at 1e-6 tolerance ? a value exactly at `x.xxxxxx5` could round differently, but since both sides go through the same `_round` it's consistent. The real cross-machine risk is in `build_candidates` producing slightly different floats (e.g. `math.sin`, `**0.5`). Let me check if any value sits right at the 1e-6 boundary.

41. user (2026-06-16T06:28:48.736Z)

```
=== schema_version guard: only > rejected (downgrade/equal accepted) ===
60:#: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
61:CURRENT_SCHEMA = 1
305:    Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
343:        "schema_version": CURRENT_SCHEMA,
378:        "schema_version": CURRENT_SCHEMA,
641:        "schema_version": CURRENT_SCHEMA,
697:        "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
732:        "schema_version": CURRENT_SCHEMA,
757:    whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
```

```
761:    sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
762:    if sv > CURRENT_SCHEMA:
764:            f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade the
harness")
777:    TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be EXACTLY
794:    exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))

=== mean_iou 0.983 ? is that >1 active_ratio a concern? float fields near 1e-6 boundary ===
active_ratio=1.008475 (>1.0 ? active_frame_count/win_frames rounding; benign but note)
```

42. assistant (2026-06-16T06:29:05.660Z)

I have a complete and well-grounded picture. Let me synthesize the findings against the ROBUSTNESS lens.

Key confirmed facts:
1. **`max_win` is pinned-but-ignored in the replay.** `provenance.windowing.max_win` and `manifest.windowing.max_win` carry `0.0`, but `_pinned_proposal()` returns only `(vt, mg, mwd)` and `build_candidates()` has no `max_window_duration` parameter. The W1 bounded-windowing split (PR #187) is therefore **structurally unreachable from the replay**. If a future PR flips SERVED's `max_window_duration > 0` (or changes the W1 split algorithm), the production windowing changes but **the fixtures stay green** ? silent pass, not a loud fail. The pinned literal is genuinely safe against SERVED churn (good), but the cost is that the W1 dimension of the windowing algorithm is uncharacterized.

2. **The doc §4a overclaims the reuse map.** It states the replay path is `parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates ? rally_gate net_crossings ? ...` and "Reuses verbatim ... `windowing.WindowingPreset` (default SERVED)". The actual code never imports or calls `windowing.build_windows`/`WindowingPreset`/`SERVED` ? it calls `distill_local.build_candidates` with a raw 3-tuple. So `windowing.py` is NOT on the replay path at all. A future refactor of `windowing.build_windows` or the `WindowingPreset` ? tuple bridge would not be caught by these fixtures, contradicting the doc.

3. **`build_candidates` signature change handling.** If `build_candidates`'s `proposal` 3-tuple shape changes (e.g. to accept `max_win` as a 4-tuple, or to take a `WindowingPreset`), the replay's `distill_local.build_candidates(..., proposal=proposal)` call breaks with a `TypeError`/`ValueError` ? that is a **loud** failure (good). If `trajectory_to_action_windows`'s *algorithm* changes (velocity formula, merge logic) the floats shift and the parse-compare fails loudly (good). If `rally_gate.estimate_play_axis`/`count_net_crossings` change, `net_crossings` (exact-int compared) fails loudly (good).

Let me confirm the remaining nuances around the net-crossings gate interaction with CONTROL and the FEATURE_NAMES assert.

The `FEATURE_NAMES` drift assert (line 82) is module-level ? if `distill_local.FEATURE_NAMES` changes, **import of golden_fixtures raises AssertionError**, which would break collection loudly. Good. The net_crossings feature IS computed in the replay and stored/compared, but CONTROL's `predict()` uses `min_crossings=0` so the gate is a no-op on segments ? meaning a change to the `rally_gate` *keep-decision* threshold wouldn't affect `control_segments`, but a change to the *crossing count* would still be caught via the exact-int `net_crossings` comparison.

I have enough to write the structured findings.

43. user (2026-06-16T06:30:16.919Z)

Structured output provided successfully

44. assistant (2026-06-16T06:30:29.472Z)

I've completed the robustness audit of the shipped Golden Regression Fixtures feature and returned my findings via StructuredOutput.

Key takeaways for the parent agent:

The big miss (medium): `max_window_duration` / W1 bounded-windowing (PR #187 ? the windowing session the lens explicitly names) is pinned in `provenance.windowing.max_win` and `manifest.json` as `0.0`, but is **structurally unreachable from the replay**. `_pinned_proposal` (`backend/eval/golden_fixtures.py:287-293`) returns only `(vt, mg, mwd)`, and `distill_local.build_candidates` (`distill_local.py:52-59`) has no `max_window_duration` parameter. So if a future PR flips SERVED's `max_window_duration > 0` (it sources from `IndexingConfig`, currently `0.0` at `config/models.py:138`) or changes the `_split_overlong_groups` algorithm (`tracknet_runner.py:334-369`), production windowing changes while the fixtures stay green ? the exact "passes silently (bad)" failure mode. This is a pinned knob the replay ignores, reading as coverage that does not exist.

The good news (the lens's main worry is safe): the most-feared interactions fail LOUDLY. `FEATURE_NAMES` drift raises at import (`golden_fixtures.py:82-84`); a `build_candidates` proposal-shape change breaks the explicit 3-tuple call (line 319); windowing math changes diff via the 1e-6 parse-compare; and net-crossings is compared exact-int even though CONTROL gates at `min_crossings=0`, so a crossing-count change (the #188/#190 gate) is caught. CONTROL touches numpy only via import, never execution.

Doc overclaim (low): `docs/GOLDEN_REGRESSION_FIXTURES.md:79-81` says the replay goes through `windowing.build_windows`/`WindowingPreset`/"default SERVED" ? none of which the code touches. The module docstring has the correct map; the doc is stale and self-contradictory with its own §4c.

Plus two forward-looking low-severity notes (schema_version has no downgrade/migration path once bumped; no row-count cap for P3 corpus growth + unvalidated `frame_width` in `freeze_real_fixture`). Verdict: ship-safe for the committed synthetic tier; fix the `max_win` gap and §4a before W1 promotion or real fixtures land.